ETH zürich

ML-Based Deep Bitstream Inspection for Serverless FPGAs

Can an FPGA Inspect Its Own Bitstreams? ML-Based Security with hls4ml

Santiago de Heredia
Systems Group, ETH Zürich
Spring Semester Project, 2026

Supervised by: Maximilian Heer
Professor: Prof. Dr. Gustavo Alonso

Agenda

1. **Background and Motivation**
2. **Prior work and project gap**
3. **Dataset and CNN inspection pipeline**
4. **hls4ml synthesis, hardware results and benchmarks**
5. **Coyote integration and takeaways**

Background: Serverless FPGAs

CONCEPT

1 FPGAs as on-demand accelerators
Users submit hardware applications for execution on datacenter FPGA infrastructure.

2 Partial reconfiguration for rapid deployment
User logic is loaded into a PR region without rebuilding the whole FPGA system.

3 Serverless abstraction
Users invoke acceleration without managing the shell, networking, or memory services.

4 Multi-tenant opportunity
A shared FPGA platform can serve multiple users or applications to improve utilisation.

Serverless FPGA

Work-In-Progress

Fabio Maschi
Systems Group, Department of Computer Science, ETH Zurich
Zürich, Switzerland

Dario Korolija
Systems Group, Department of Computer Science, ETH Zurich
Zürich, Switzerland

Gustavo Alonso
Systems Group, Department of Computer Science, ETH Zurich
Zürich, Switzerland

F3: An FPGA-accelerated FaaS Framework

Charalampos Mainas
Technical University of Munich
Munich, Germany

Martin Lambeck
Technical University of Munich
Munich, Germany

Bruno Scheufler
Technical University of Munich
Munich, Germany

Laurent Bindschaedler
Max Planck Institute for Software Systems (MPI-SWS)
Saarbrücken, Germany

Atsushi Koshiba
Technical University of Munich
Munich, Germany

Pramod Bhatotia
Technical University of Munich
Munich, Germany

Coyote v2: Raising the Level of Abstraction for Data Center FPGAs

Benjamin Ramhorst*
benjamin.ramhorst@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

Dario Korolija*†
dario.korolija@amd.com
AMD Research
Zurich, Switzerland

Maximilian Jakob Heer
maximilian.heer@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

Jonas Dann
jonas.dann@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

Luhao Liu‡
luhliu@student.ethz.ch
ETH Zurich
Zurich, Switzerland

Gustavo Alonso
alonso@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

Motivation: Serverless FPGAs Threat Model → Why inspect bitstreams?

WHY THIS MATTERS

1 Shared FPGA capacity

Serverless FPGA platforms aim to share reconfigurable fabric and platform services across users.

2 Untrusted tenant logic

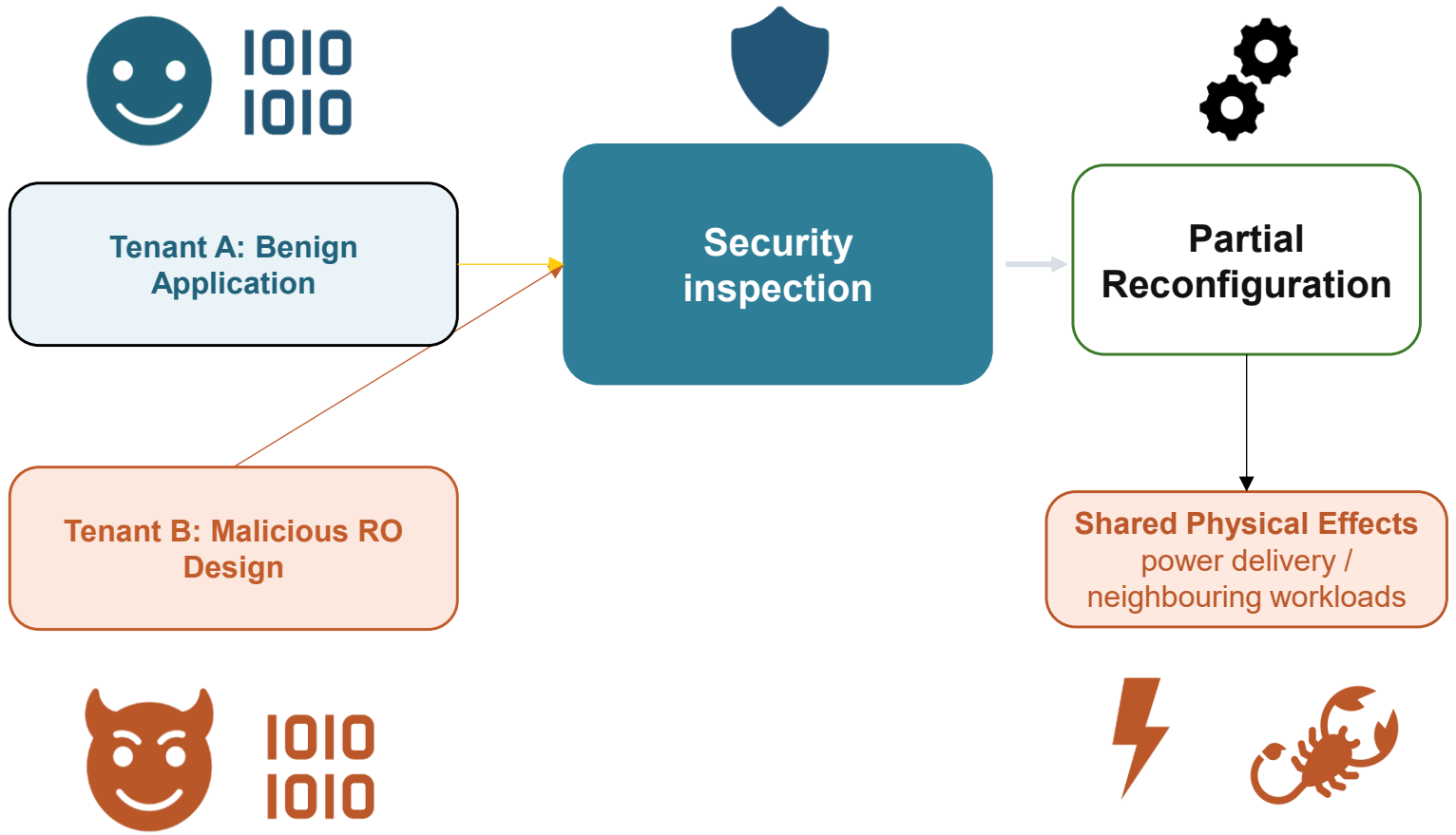
A submitted partial bitstream can contain circuits that stress / exploit shared physical resources.

- power delivery
- neighboring tenants
- availability
- side-channels

3 Gate before deployment

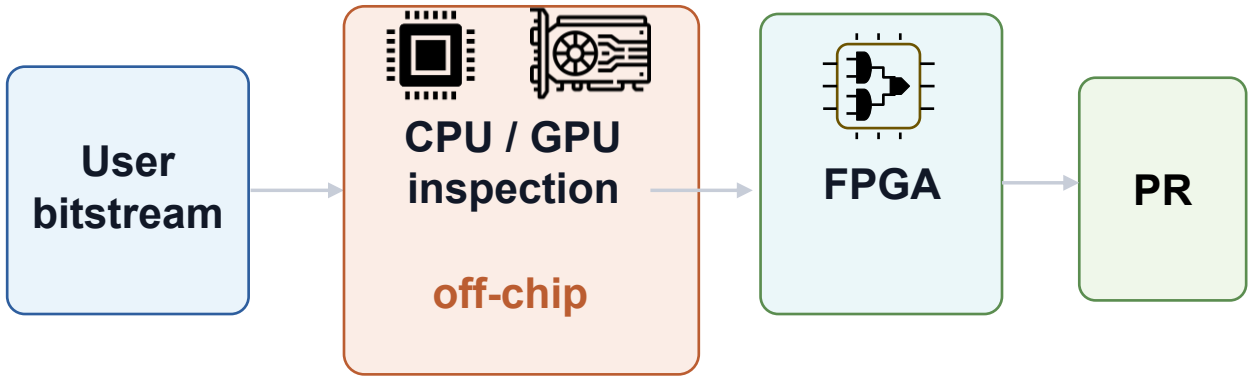
Inspect each user partial bitstream before it enters the PR path.

DEPLOYMENT PATH



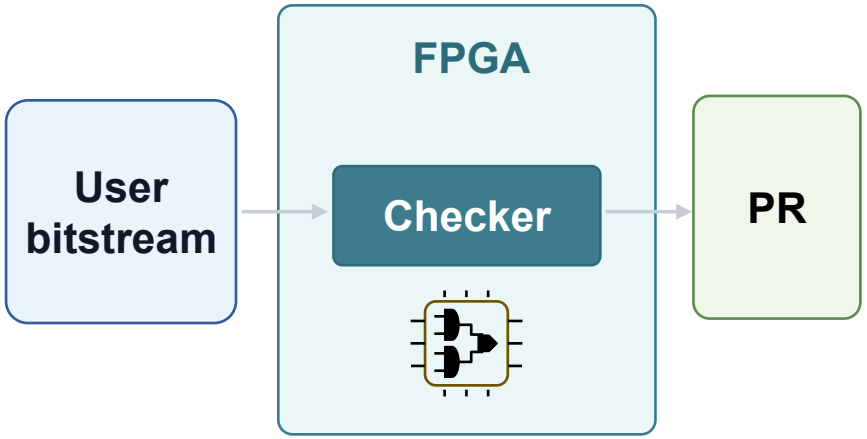
Motivation: Why FPGA-resident inspection?

HOST-SIDE INSPECTION



extra host path + orchestration

FPGA-RESIDENT INSPECTION



inspection stays in PR path

Latency / data movement
avoid unnecessary off-chip inspection transfers

Economics
avoid occupying CPU / GPU resources for each check

Systems simplicity
one self-contained FPGA deployment path

Goal: make **serverless FPGA accelerator-as-a-service more feasible** by moving inspection into the FPGA system itself.

Prior Work: Detecting Malicious Bitstreams

Existing methods detect malicious FPGA bitstreams, but deployment-oriented ML inspection remains the gap.

Approach	Strength	Limitation / gap
Cloud / Vendor checks [1]	Practical, integrated into deployment flows	Limited to known rule violations
Structural / RO-specific analysis [2]	Targets suspicious oscillator structures directly	Can be difficult to generalize or scale
ML-based inspection [3,4]	Learns malicious bitstream patterns	Generally evaluated as offline inference
This work	**ML inspection of partial bitstreams, synthesized for Coyote-style (serverless FPGAs) deployment**	**Feasibility prototype for online deployment-path inspection**

[1] La et al., “Denial-of-Service on FPGA-Based Cloud Infrastructures—Attack and Defense,” TCHES, 2021.
[2] La et al., “FPGADefender: Malicious Self-Oscillator Scanning for Xilinx UltraScale+ FPGAs,” ACM TRETS, 2020.
[3] Elnaggar et al., “Learning Malicious Circuits in FPGA Bitstreams,” IEEE TCAD, 2023.
[4] Chaudhuri and Chakrabarty, “Detection of Malicious FPGA Bitstreams Using CNN-Based Learning,” ETS, 2022.

Research Gap: From Offline Detection to FPGA-Resident PR Control

Can ML-based bitstream inspection run directly on the FPGA deployment path?

Three dimensions define whether the idea is feasible

Detection

Does it retain useful security coverage?

Evaluate: F1 · FNR · FPR · AUC

Relevance: classify malicious vs. benign partial bitstreams

Hardware cost

Can it fit beside platform and user logic?

Evaluate: latency · LUT · BRAM · DSP

Relevance: meet deployment latency and area limits

Integration

Can it sit on the Coyote PR path?

Evaluate: Coyote-style PR-path prototype

Relevance: enable self-contained checking

Takeaway: success requires detection quality, hardware feasibility, and a concrete deployment path.

Contributions / Differentiators

1. Partial-Bitstream Dataset

Partial bitstreams generated from **real Coyote applications**, complemented by RO-based malicious payloads.

2. Datacenter FPGA setting

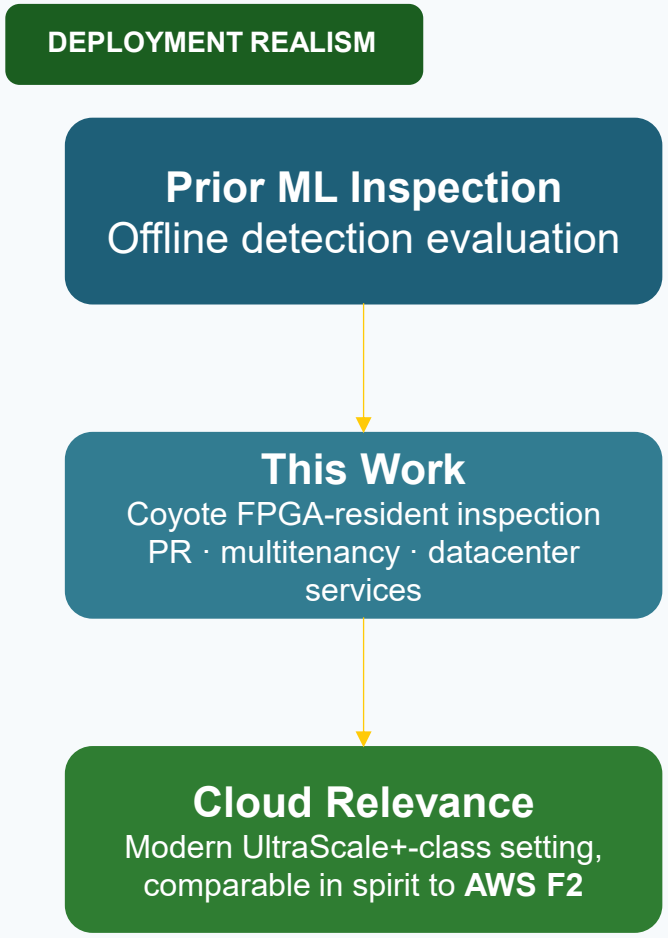
Inspection evaluated in a **Coyote shell** with **PR**, **multitenancy**, and modern **UltraScale+ hardware**.

3. ML-to-hardware path

Connects CNN training to hls4ml synthesis and concrete latency/resource evaluation.

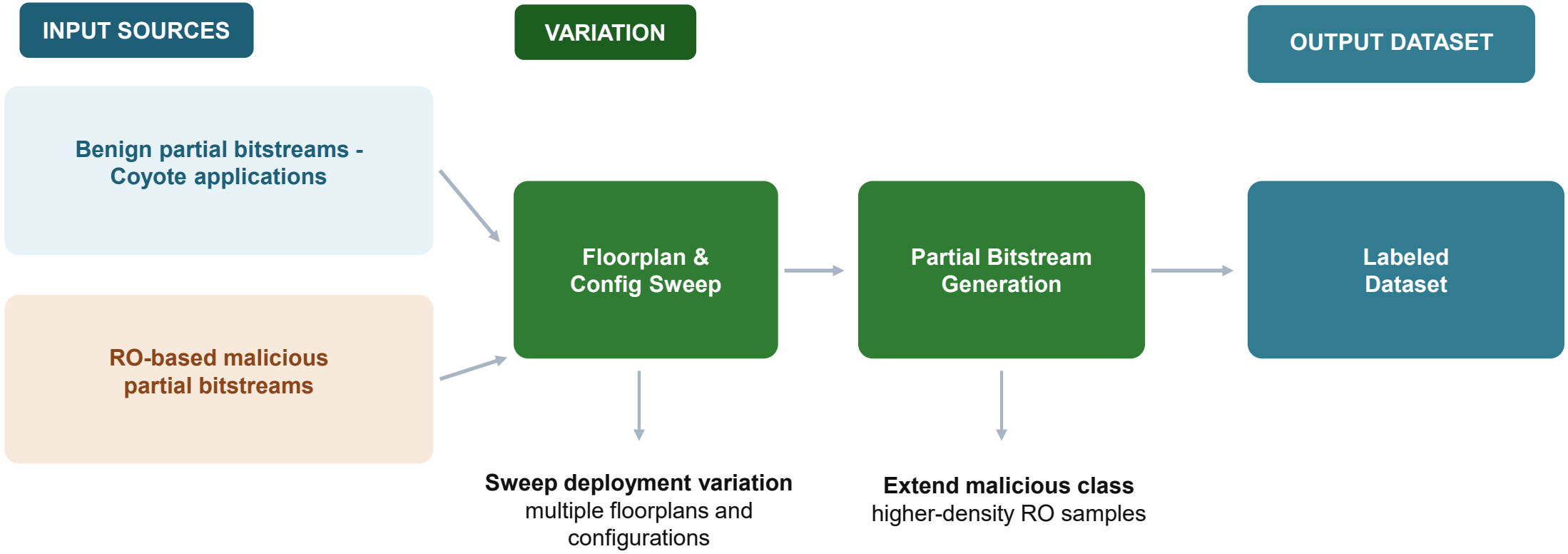
4. Design-space evaluation

Compares accuracy, FNR/FPR, AUC, LUT, BRAM, DSP, and latency across candidates.



Dataset Generation Pipeline

Construct benign and malicious partial bitstreams with deployment-like variation



Goal: evaluate deployment-like artifacts with real Coyote applications, and strong RO payloads

Model Selection: CNNs + Justification

PRIOR EVIDENCE

Bitstreams contain learnable structure

Prior work identifies modules/functions from bitstream-derived representations. [1,2]

CNNs/ML can operate directly on bitstream data

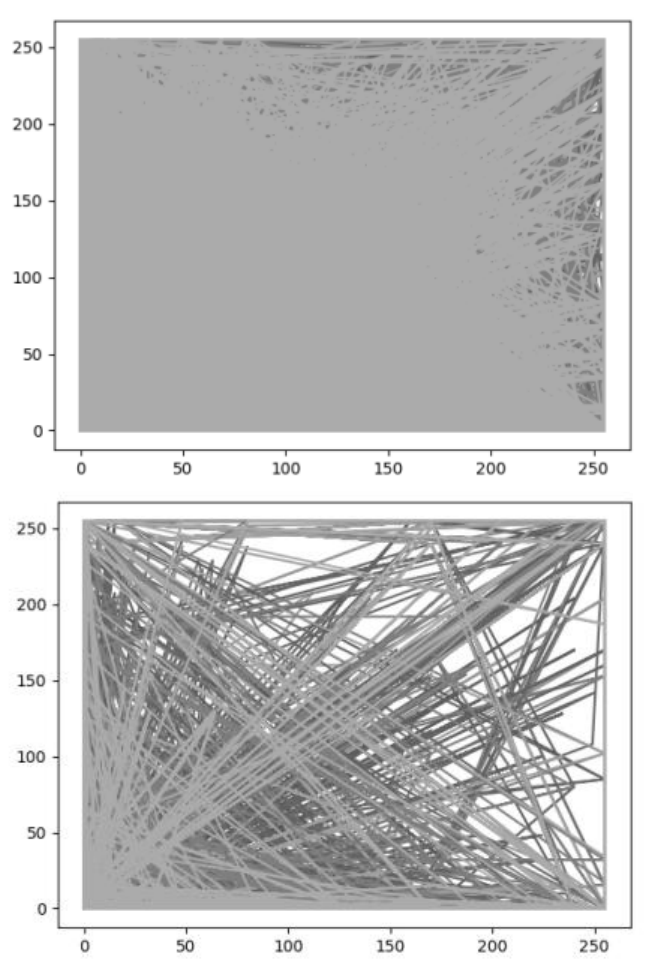
Models learn spatial/statistical patterns without semantic decoding of the circuit. [1,2]

This has been applied to security

ML/CNN approaches have been used for malicious bitstreams, including RO-related payloads. [3,4]

Takeaway: bitstream-as-data is a supported, practical representation for ML-based inspection.

EXAMPLE



Bitstream image representations from [4]: (top) benign and (bottom) malicious design.

[1] Puschner et al., “Deep Learning for Reverse Engineering of FPGA Bitstreams,” 2020.
[2] Rettkowski et al., “Inspection of Partial Bitstreams for FPGAs Using Artificial Neural Networks,” IPDPSW, 2019
[3] Elnaggar et al., “Learning Malicious Circuits in FPGA Bitstreams,” *IEEE TCAD*, 2023.
[4] Chaudhuri and Chakrabarty, “Detection of Malicious FPGA Bitstreams using CNN-Based Learning,” ETS, 202

Our Preprocessing Solution

A **deterministic byte-stream transform** that can run **quickly** and identically in software and FPGA logic

INPUT REPRESENTATION & RESIZING

Partial bitstream
incoming PR region

Byte stream
[b0, b1,…, bn]

No bitstream reverse engineering
No hand-crafted features

Uniform resize
downsample or pad

Simple operations that are feasible before deployment

Fixed tensor
$128^2 \cdot 256^2 \cdot 512^2$

One shape expected by the synthesized CNN

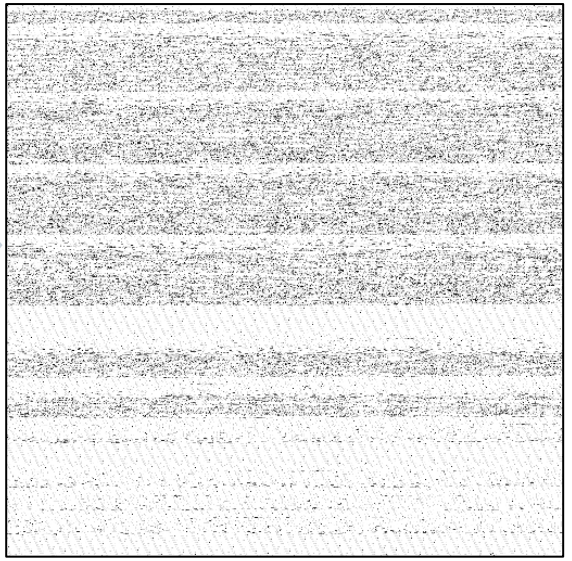
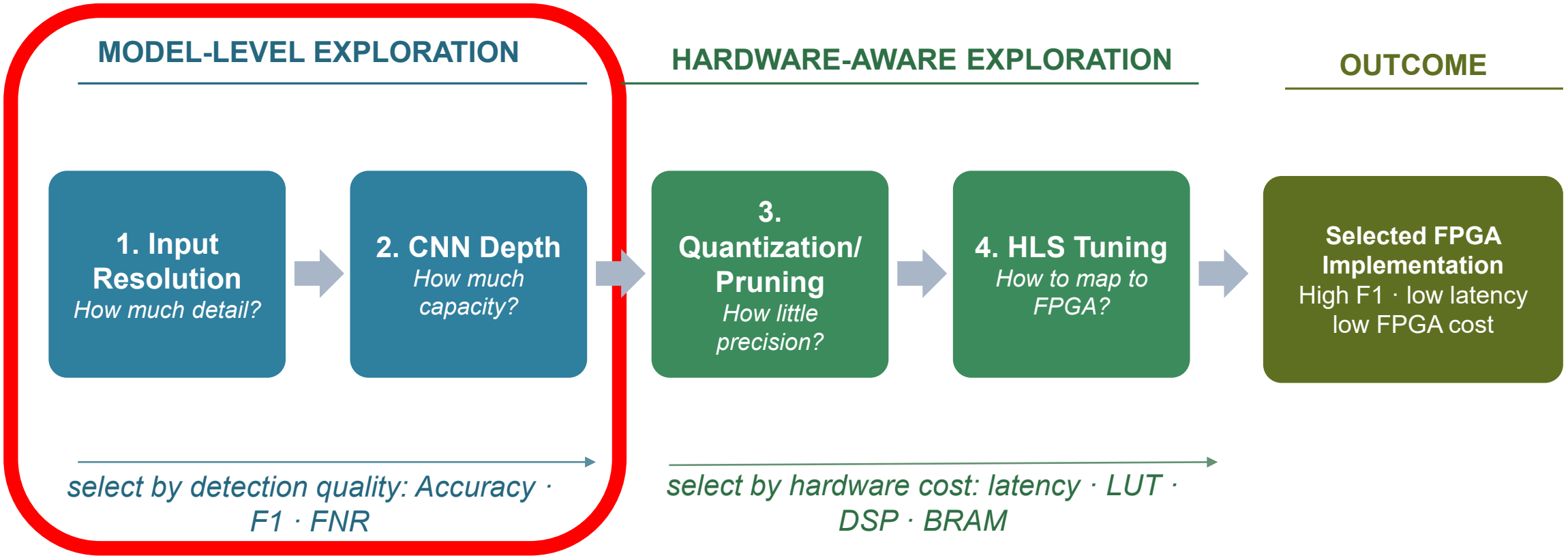


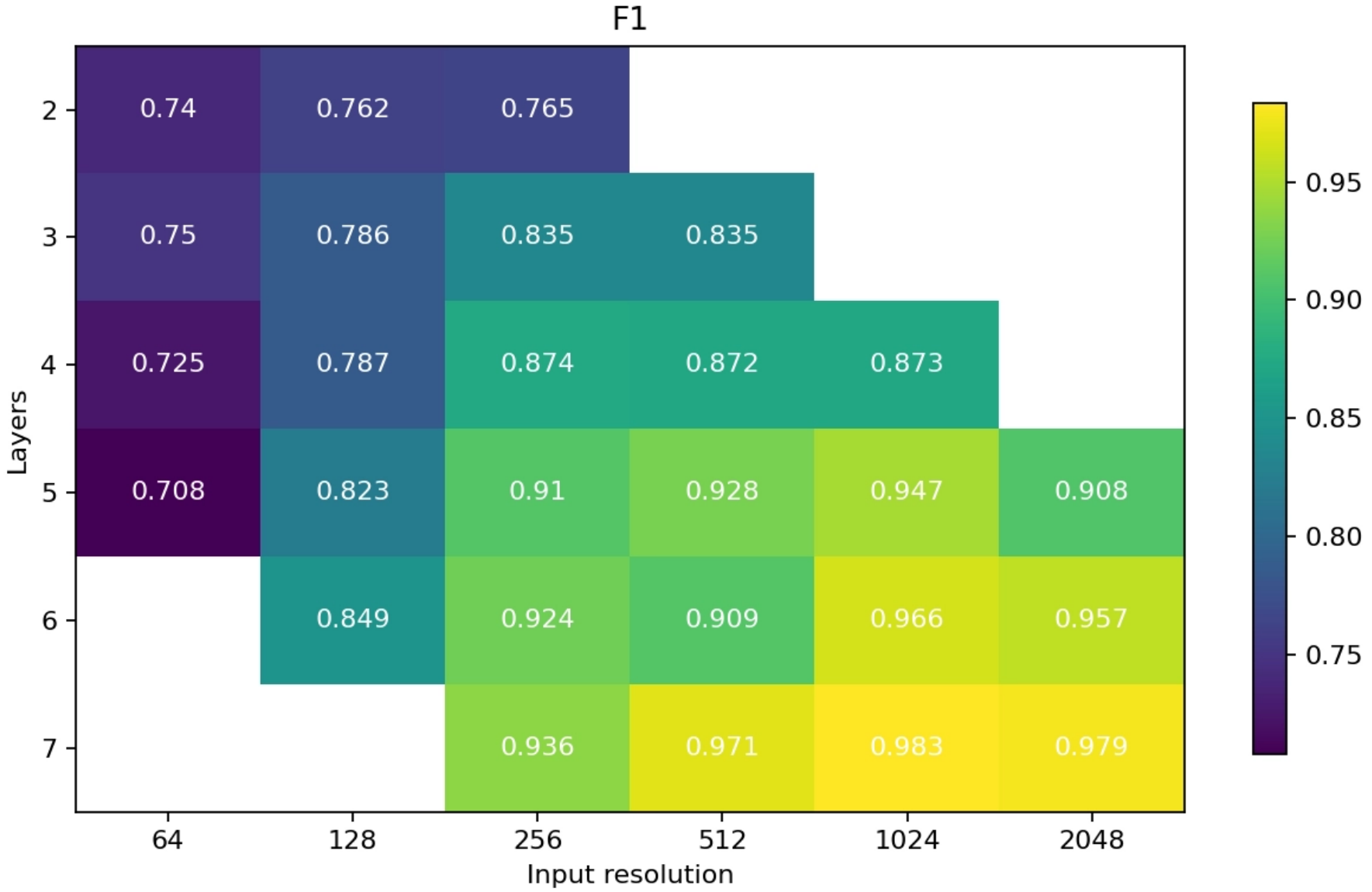
Image Representation

CNN Design Space

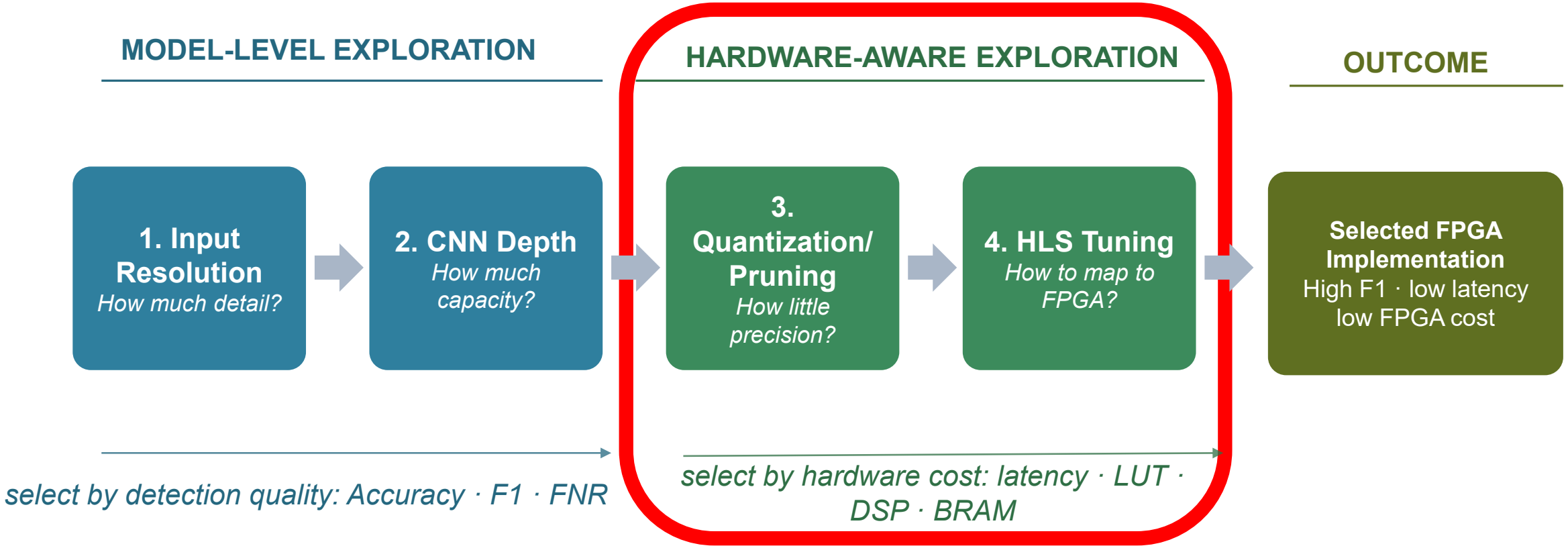


Goal: preserve detection quality while keeping latency and resource cost low .

Model-Level Exploration: F1



CNN Design Space

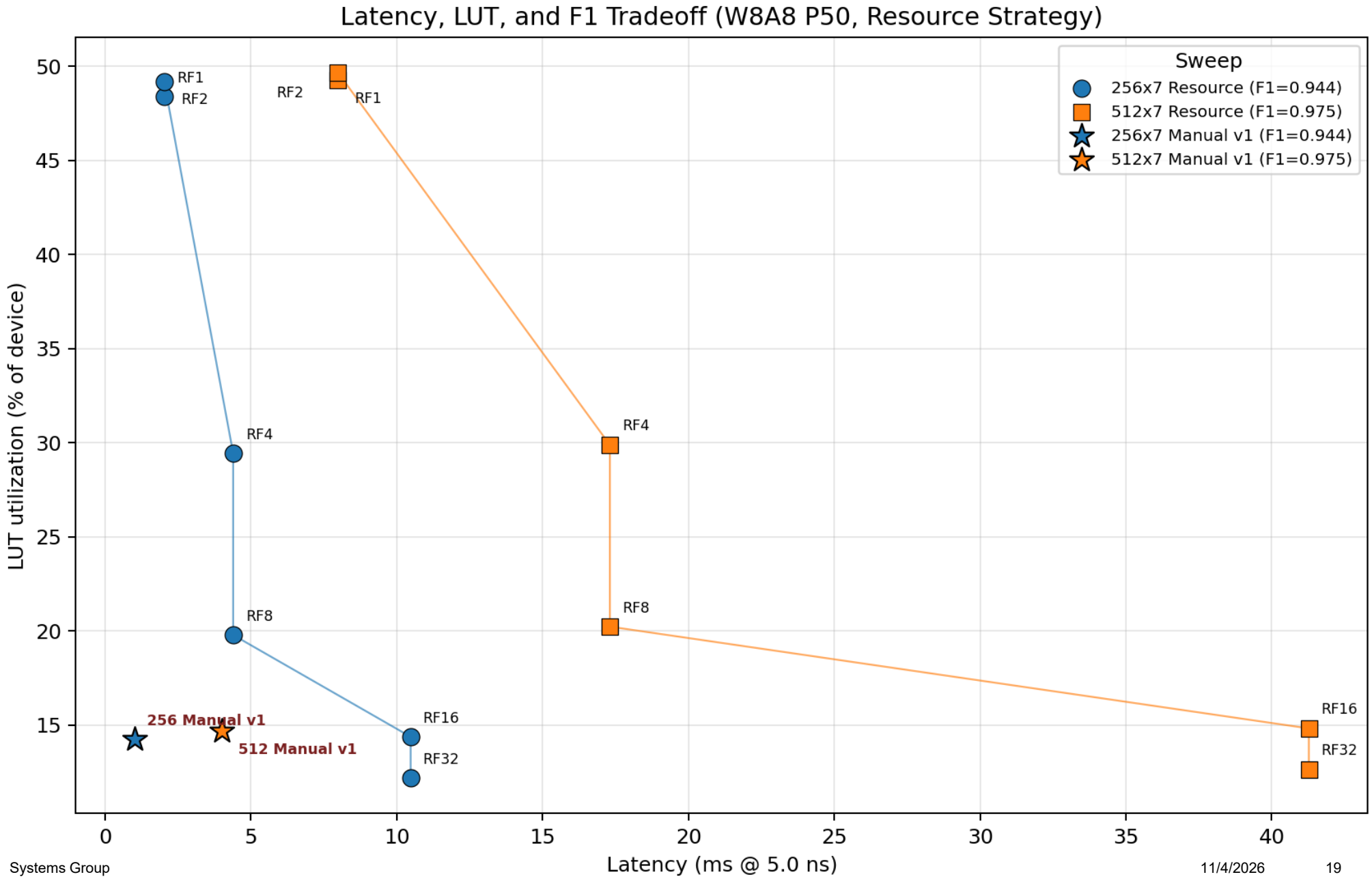


Goal: preserve detection quality while keeping latency and resource cost low .

HLS Exploration: Latency vs. Resource vs. Accuracy [F1] Trade-off

HLS Knobs

- Quantization
- Reuse factor
- Synthesis strategy
- **Manual per-layer tuning**



Synthesized Results

Synthesized Results: Accuracy + Resources

Quantized + pruned model with an optimized streaming implementation

DETECTION QUALITY

88.5%

F1

92.0%

TPR: Original

98.7%

TPR: Extended‡

Accuracy	FPR	ROC AUC
87.7%	**16.9%**	**95.1%**

Original test set unless marked ‡.
‡ Extended RO-footprint cross-validation.

FPGA RESOURCES

2.70%

BRAM

54.5 tiles

7.18%

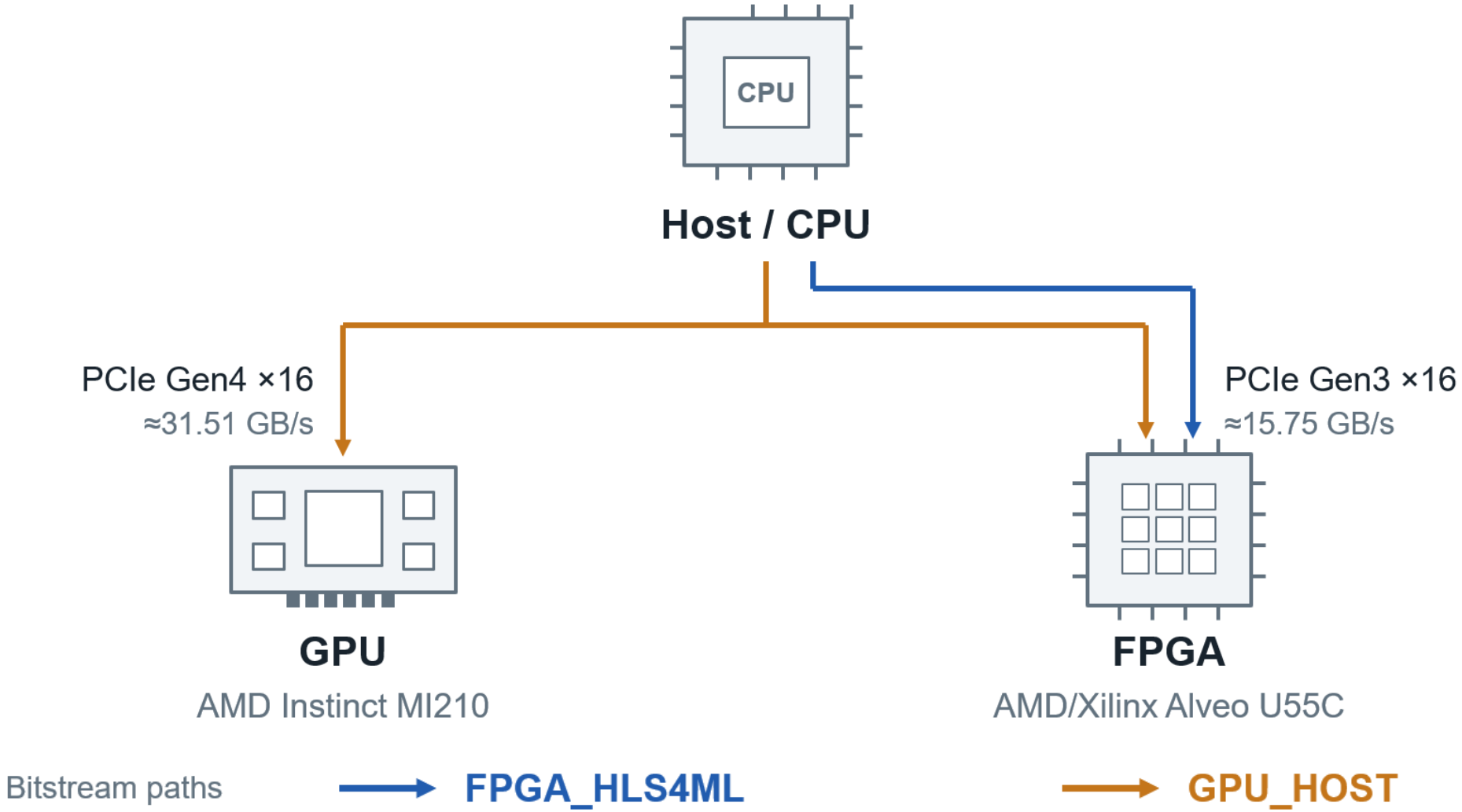
DSP

648 DSPs

Resource	Count	U55C share
LUT	**61,141**	**4.69%**
URAM	**0 blocks**	**0%**

Routed wrapper, including interfaces and ILA.
Percentages use total U55C capacity; shell excluded.

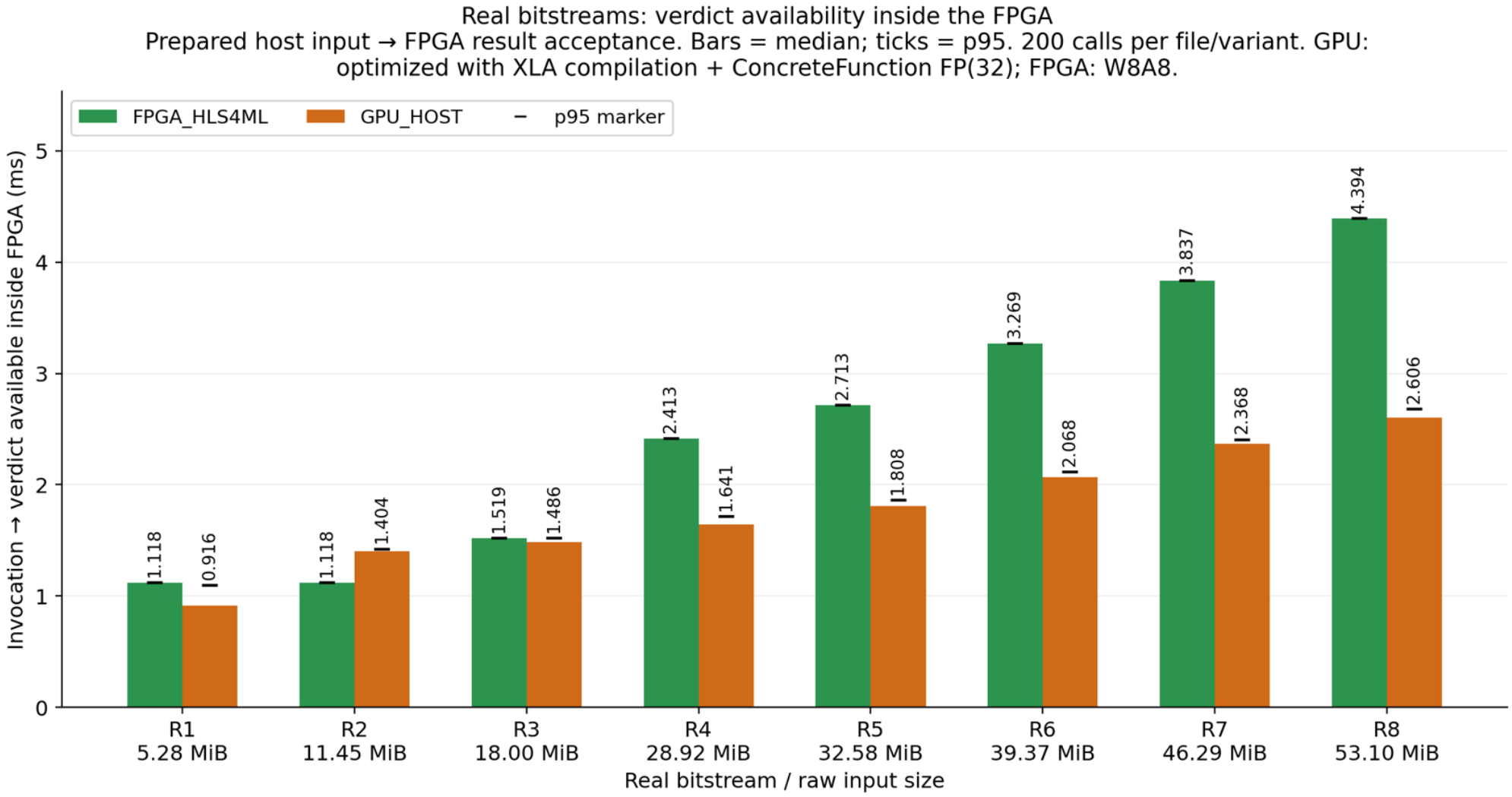
Synthesized Results: Latency Benchmark Setup



Synthesized Results: Latency

Compared against to the optimized GPU case

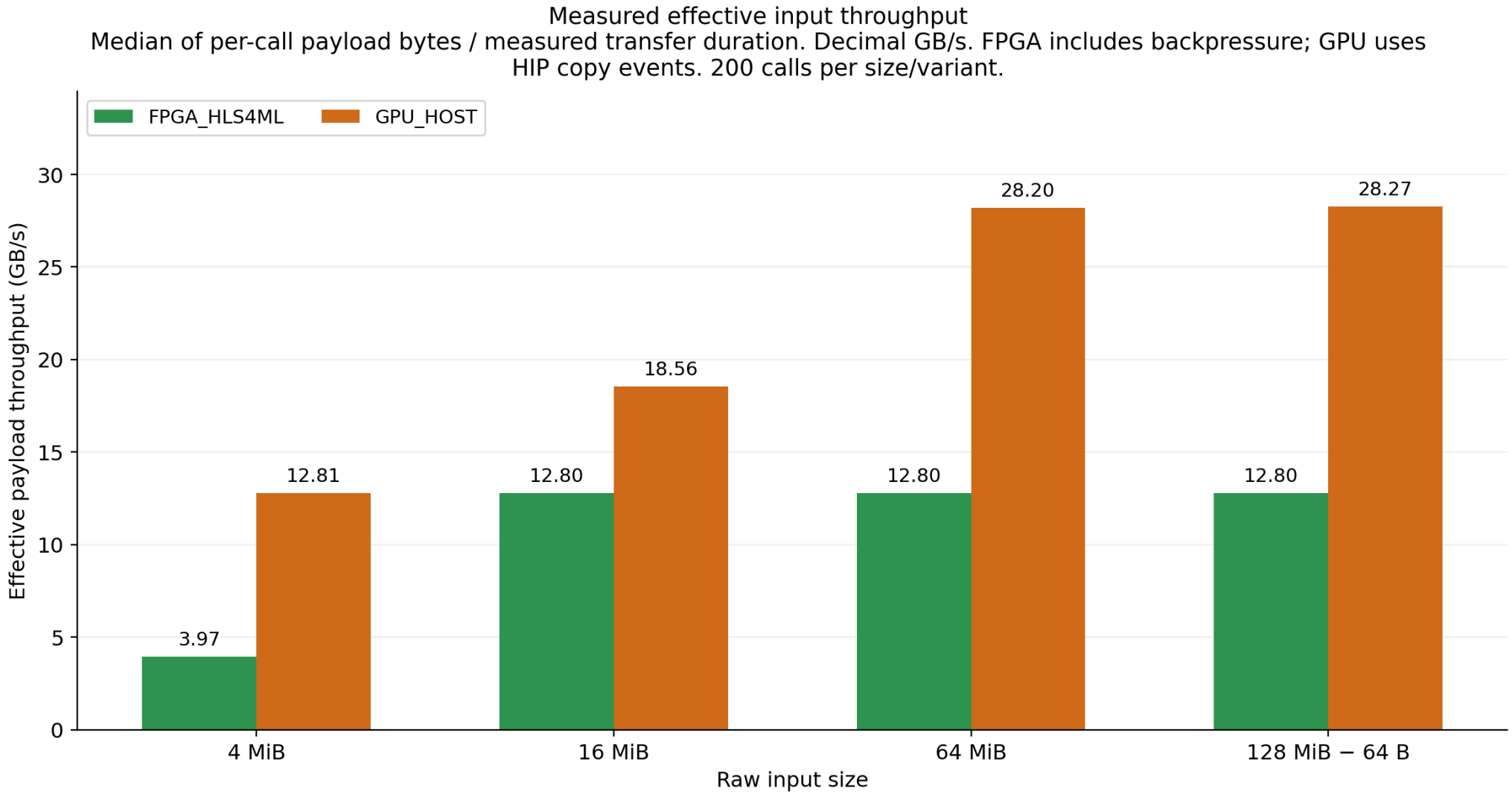
- Latency ranges from **1.1ms - 4.4ms**
- FPGA performance ranges from **1.26x faster to 1.69x slower**
- This is mostly down to our current setup **not using both PCIe blocks in the U55C**



Synthesized Results: Latency

Cause: Difference in transport time / payload throughput

- Using both PCIe blocks on the U55C with Gen4 8x would bring our theoretical throughput in par with that of the GPU at **~32GB/s**



Latency results contextualized

Inspection at the median benchmark size* adds an estimated 1.92% to the F3 warm path

2.713 ms

For our median benchmark bitstream size*

Reference	Baseline	With inspection†	**Increase†**
F3 warm invocation with reconfiguration [1]	**141 ms**	**143.71 ms**	**+1.92%**
Coyote v2 shell PR #2: network / kernels [2]	72.3 ms	75.01 ms	**+3.75%**
Coyote v2 shell PR #3: traffic sniffer [2]	85.5 ms	88.21 ms	**+3.17%**
Coyote v2 ICAP at 32.58 MiB [2] (lower bound on PR time)	≈42.70 ms	≈45.42 ms	**≈+6.35%**

* We use our median benchmark bitstream size (30.75 MiB), rounded up to the nearest measured size (32.58 MiB).

† Estimate: reference + inspection, assuming no overlap

Coyote shell PR: services + applications; kernel time excludes disk I/O and buffer copy.

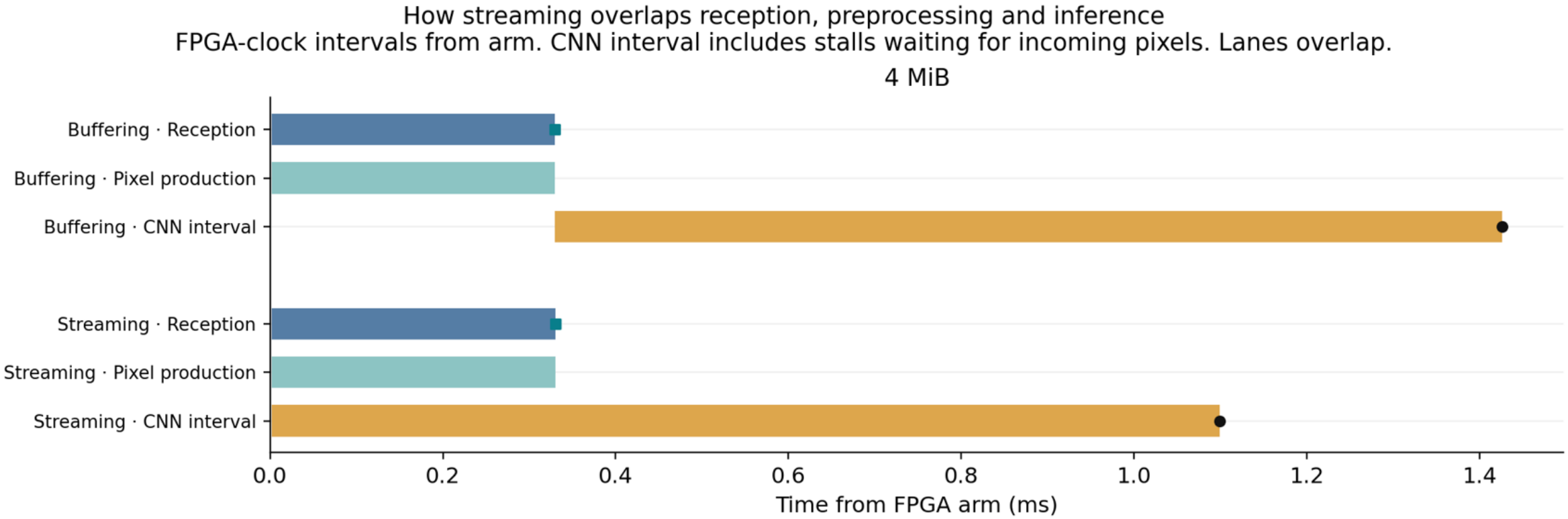
ICAP model: 34,162,708 bytes / 800 MB/s, using the paper's UltraScale+ rate; fixed controller/driver overhead excluded.

[1] Mainas et al., F3, HPDC 2025, Table 4. [2] Ramhorst et al., Coyote v2, SOSP 2025, §5.3 / Table 2 and §9.3 / Table 3.

Optimizations: Latency and Resources

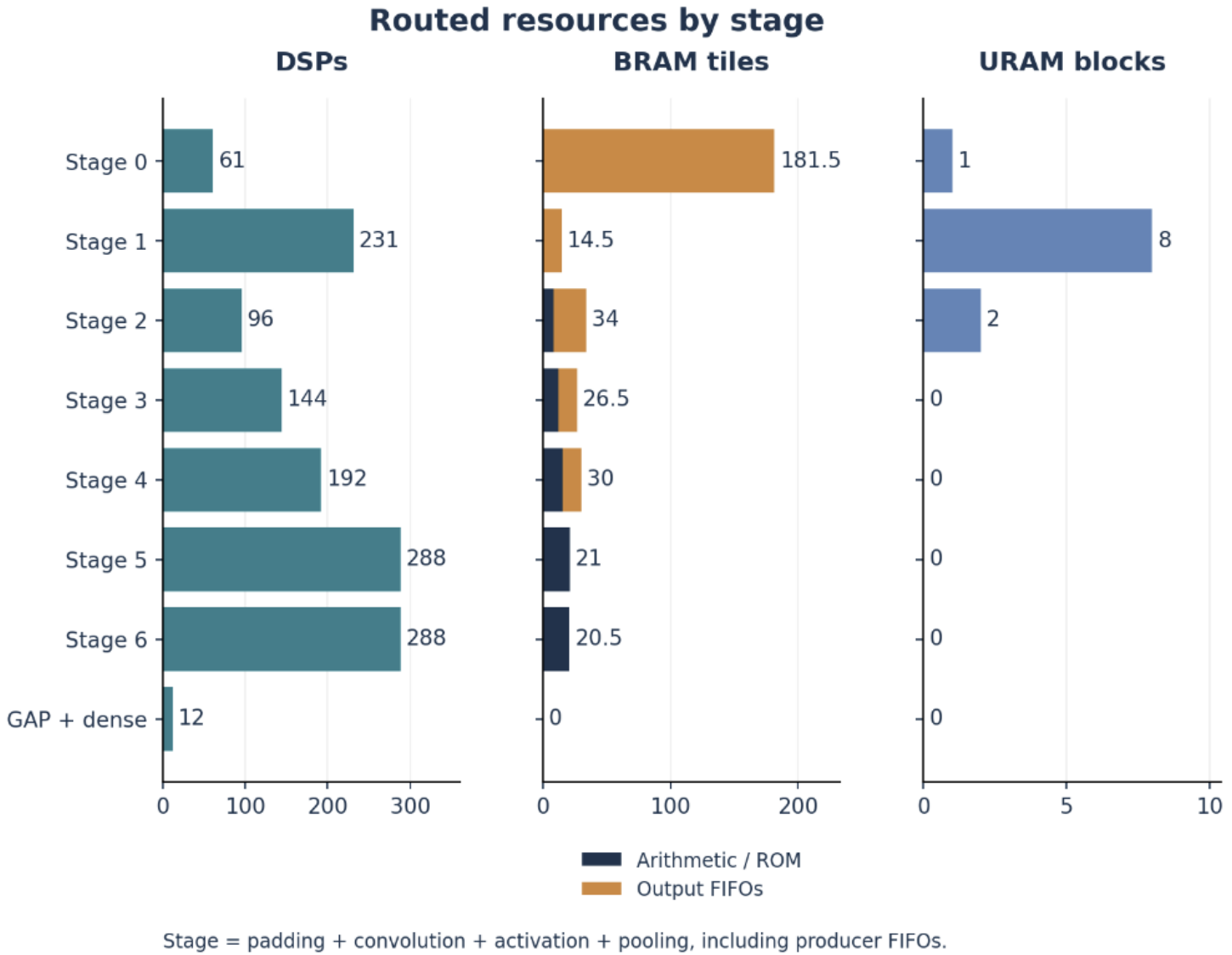
Optimizations - Latency: Streaming

- Instead of buffering preprocessed input and waiting until entire input arrives, make use of **io_stream + DATAFLOW**
- New bottleneck: **transport / reading throughput**

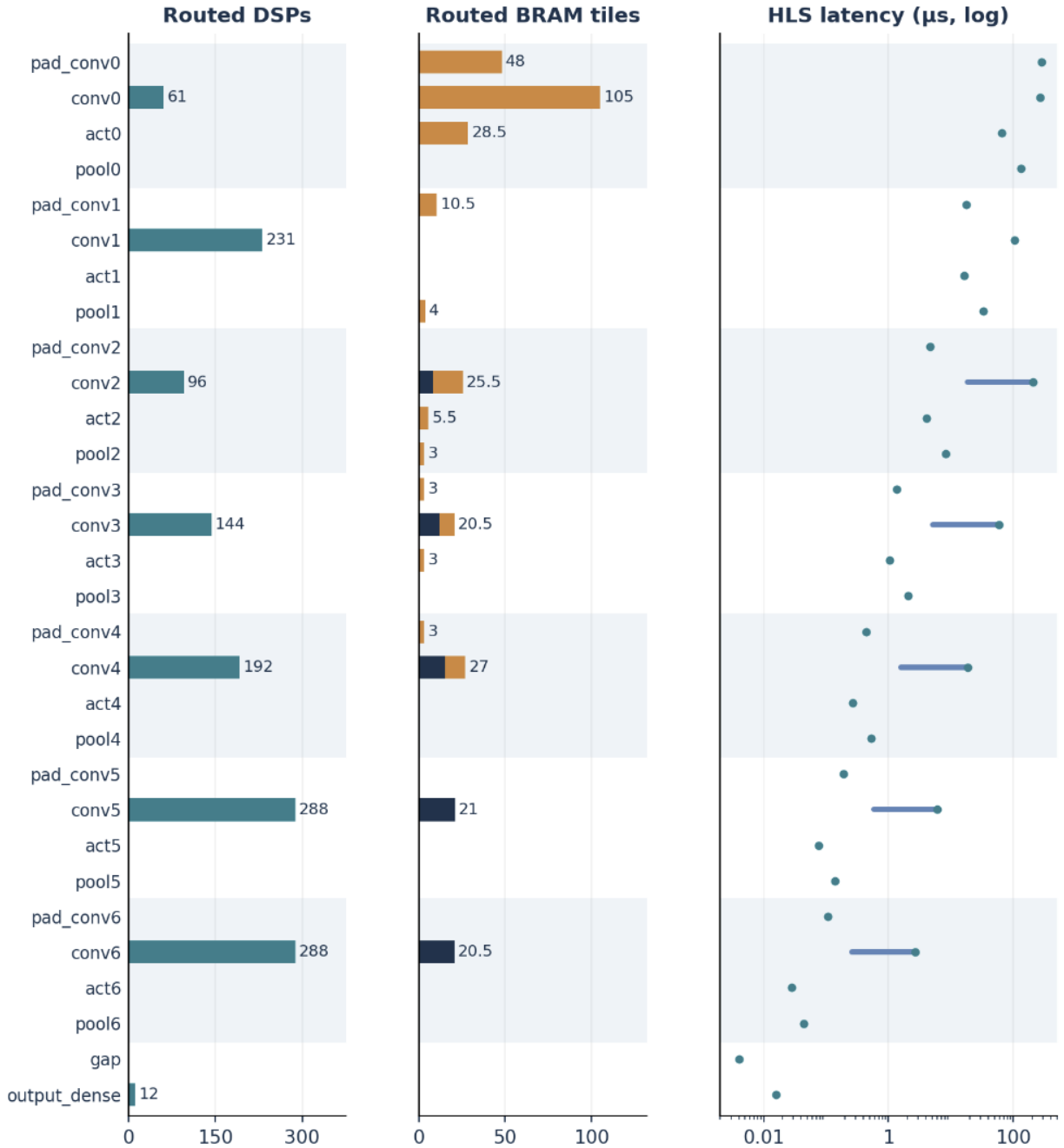


Optimizations – Resources

- Different layers of a CNN have vastly different **amounts** and **types** of resource utilization, as well as different **latencies**
- This gives room for optimizing resources + latencies **layer-by-layer**



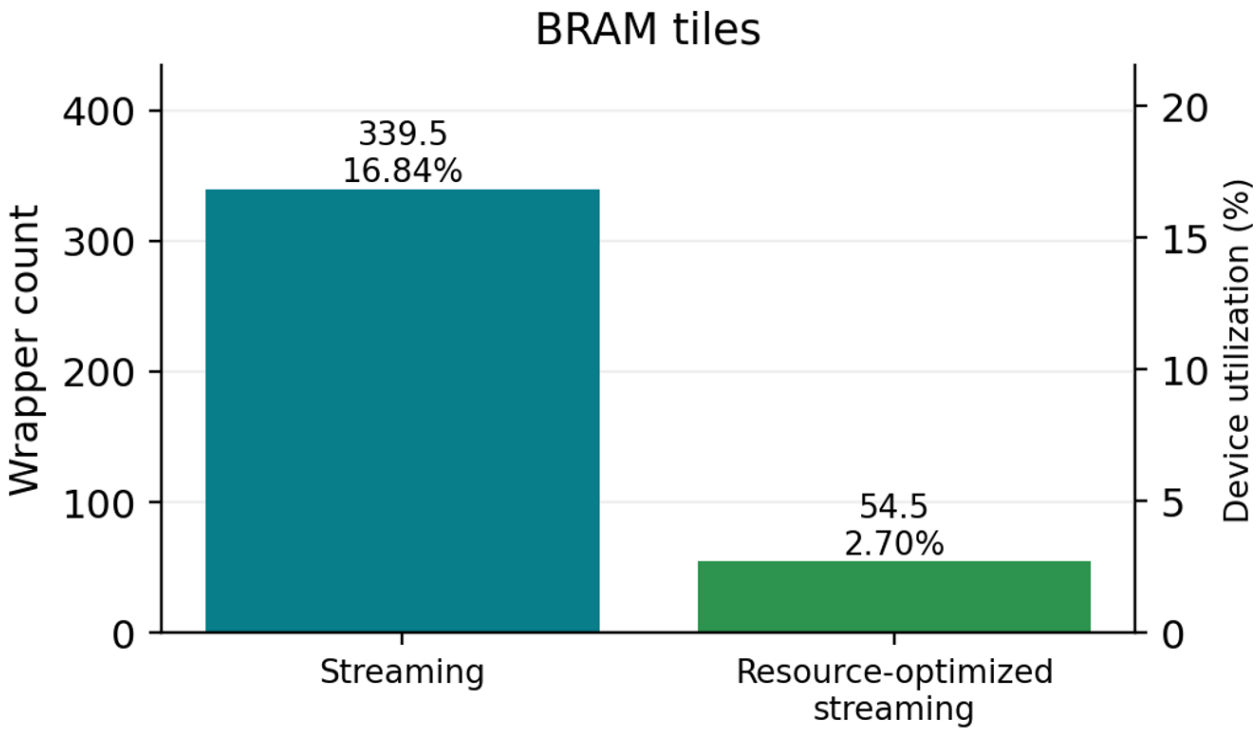
Initially unoptimized synthesized CNN, with resource attribution per-layer



Initially unoptimized synthesized CNN, with resource attribution per sub-layer and with latency attribution

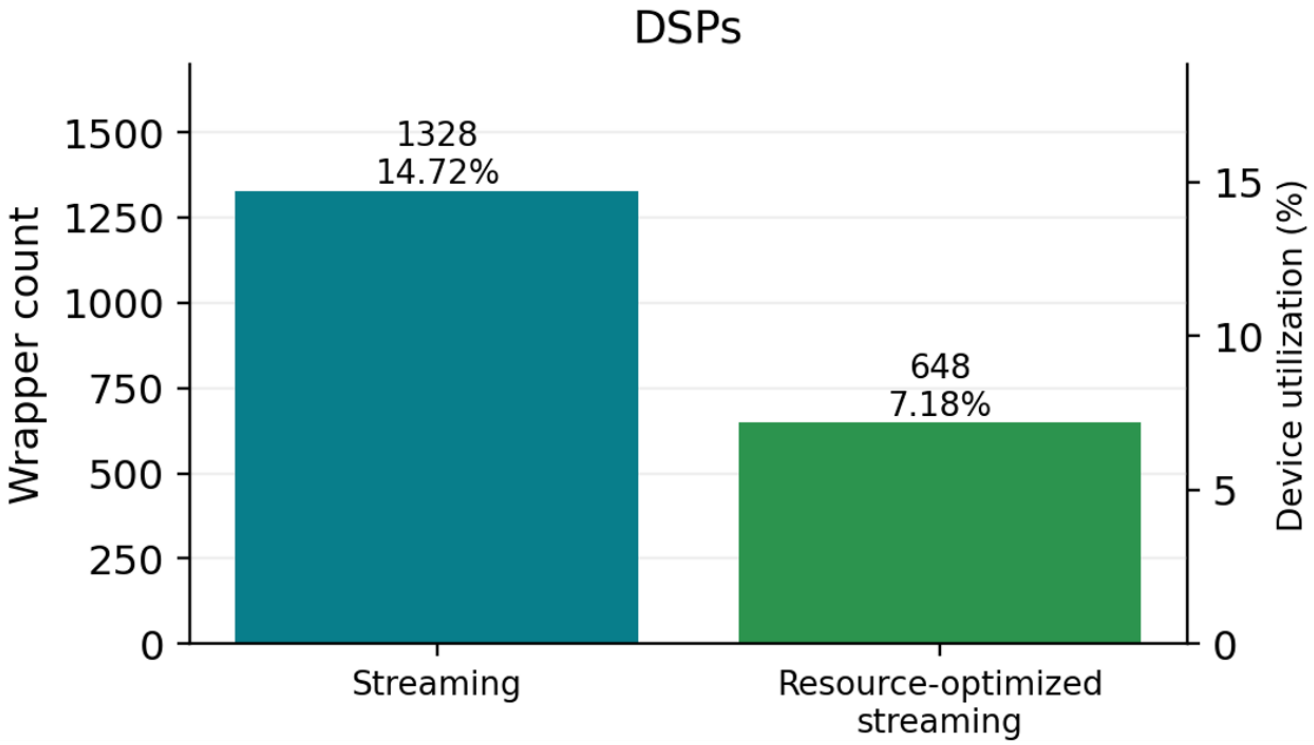
Optimizations – Resources[BRAM]: Minimising FIFOs

- Due to our large first convolutional layer, we had by default large FIFOs (depts of up to 67600 slots) which increased BRAM usage
- However, our input arrives slowly through PCIe, so these FIFO sizes can be reduced
  - Idea: Progressively reduce FIFO size and check e2e latency stays the same during benchmark

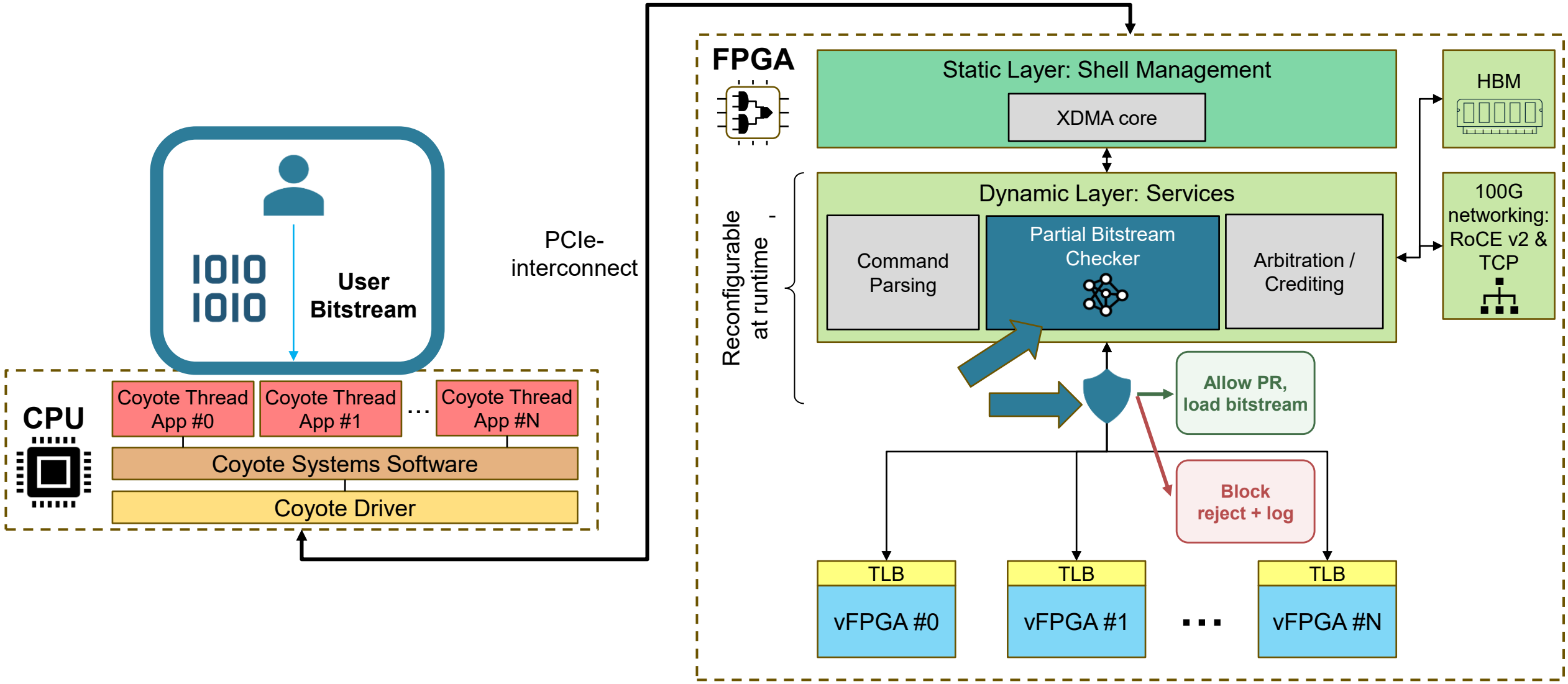


Optimizations – Resources[DSPs]: Increased Reuse Factor

- We had large amounts of DSPs in the final layers of the CNN, however these final layers barely contirbute to the latency.
  - Solution: Increase RF in later layers: **conv3/4/5/6 RF →72/72/288/288**



Proposed: Coyote Integration Path



Limitations and Future Work

Feasibility prototype → next step is broader validation, full integration w/ Coyote, and hardening

CURRENT LIMITATIONS

1 Binary classification
Current task is benign vs. RO-based malicious bitstreams.

2 CNN-only evaluation
The inspector was evaluated only with CNN-based detectors.

3 One platform family
Evaluation focused on one platform/family and Coyote-style artifacts.

4 Limited Benchmarks
Currently not using entire FPGA bandwidth nor measuring an end-to-end serverless execution

5 Partial Coyote Integration
Inspection is not yet fully in the deployment path

FUTURE WORK

1 More attack classes
Extend beyond RO payloads to richer malicious-circuit categories.

2 Explore alternative models
Benchmark other FPGA-synthesizable classifiers, such as MLPs and decision trees, alongside low-latency CNNs.

3 Generalization tests
Evaluate across devices (e.g. Versal) , floorplans, shells, and benign applications.

4 More Representative Benchmarks
Evaluate on a serverless FPGA framework (e.g. F3)

5 In-path integration
Check bitstreams directly before reconfiguration

Conclusion

Serverless FPGA-resident ML bitstream inspection is feasible

ML-based checking can move from offline analysis toward a hardware-backed Coyote deployment path.

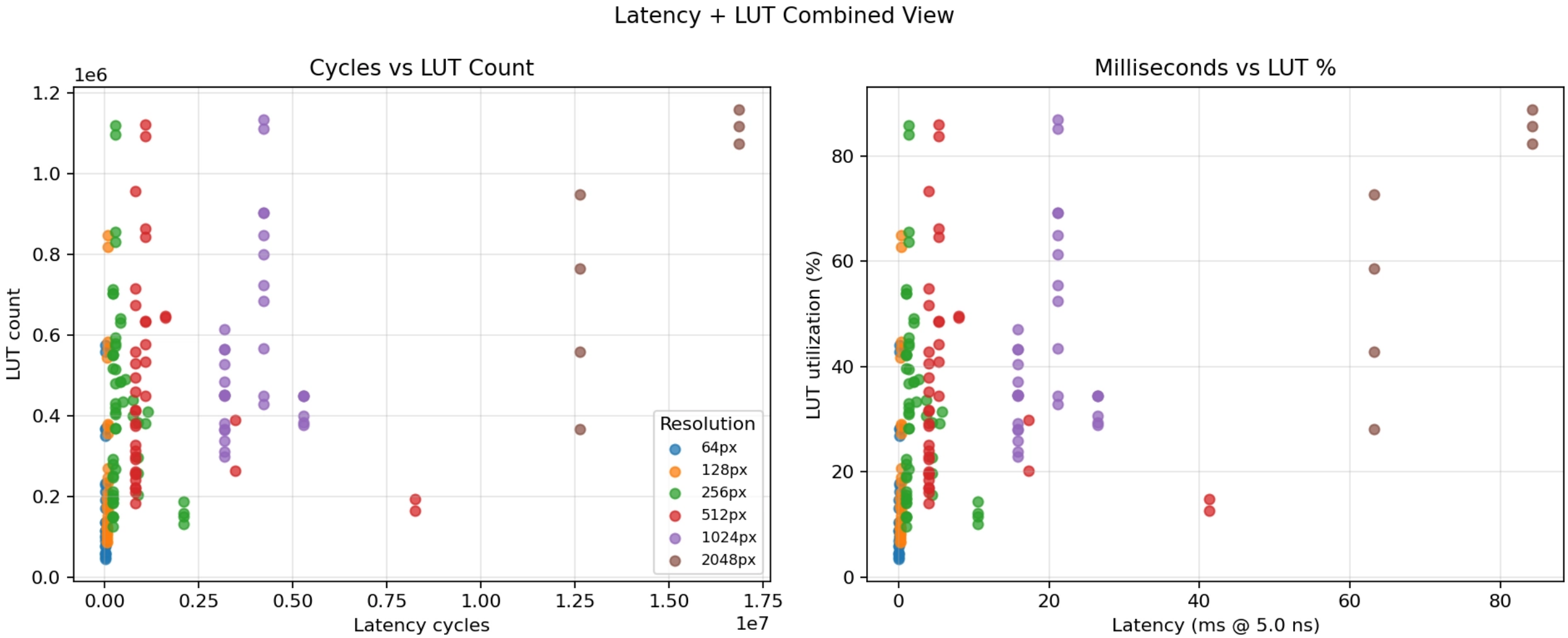
WHAT WE SHOWED

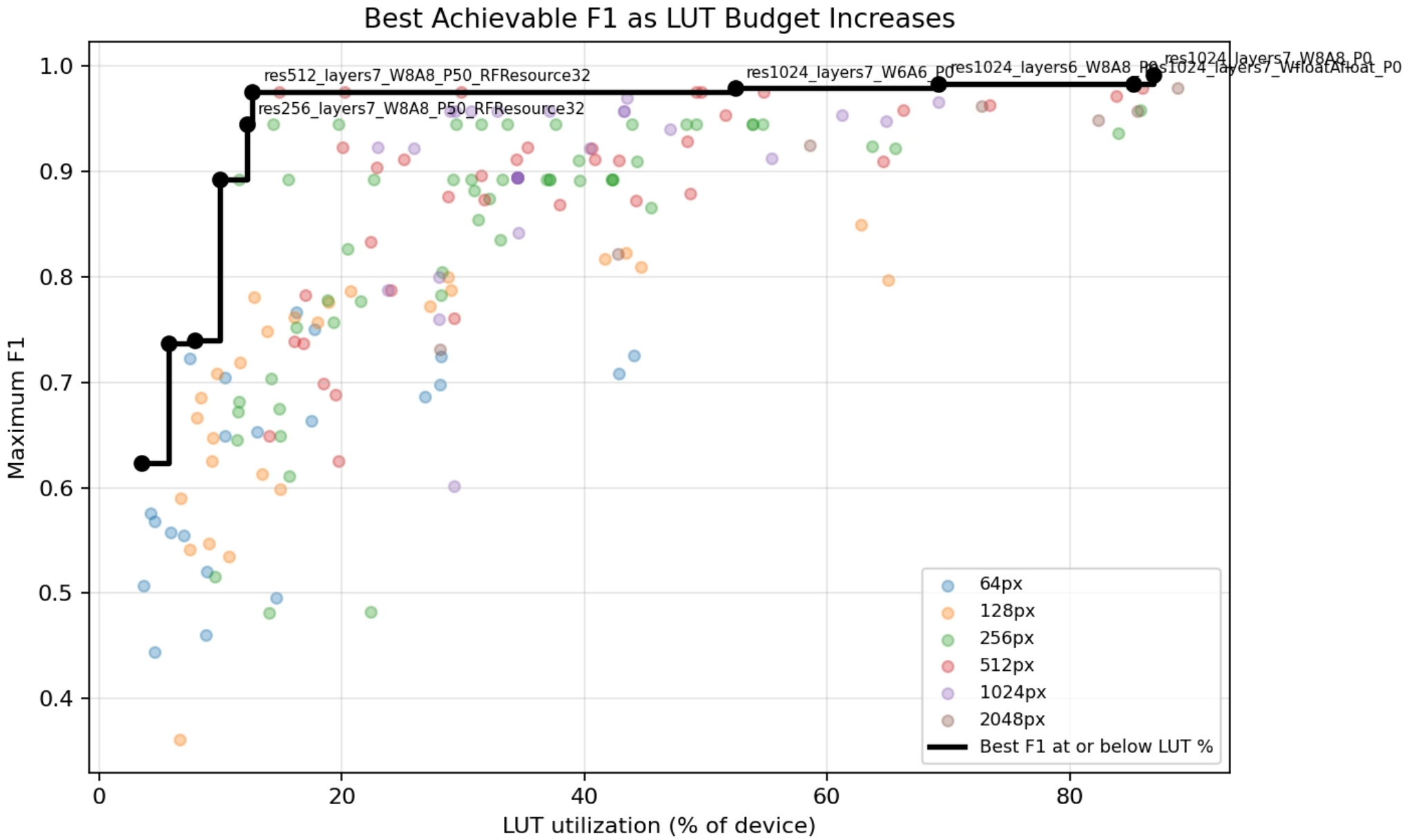
1. **Deployment path:** Inspection can be placed before partial reconfiguration.
2. **Detection quality:** CNNs classify partial bitstreams with useful accuracy.
3. **Hardware feasibility:** hls4ml synthesis gives concrete latency/resource trade-offs.

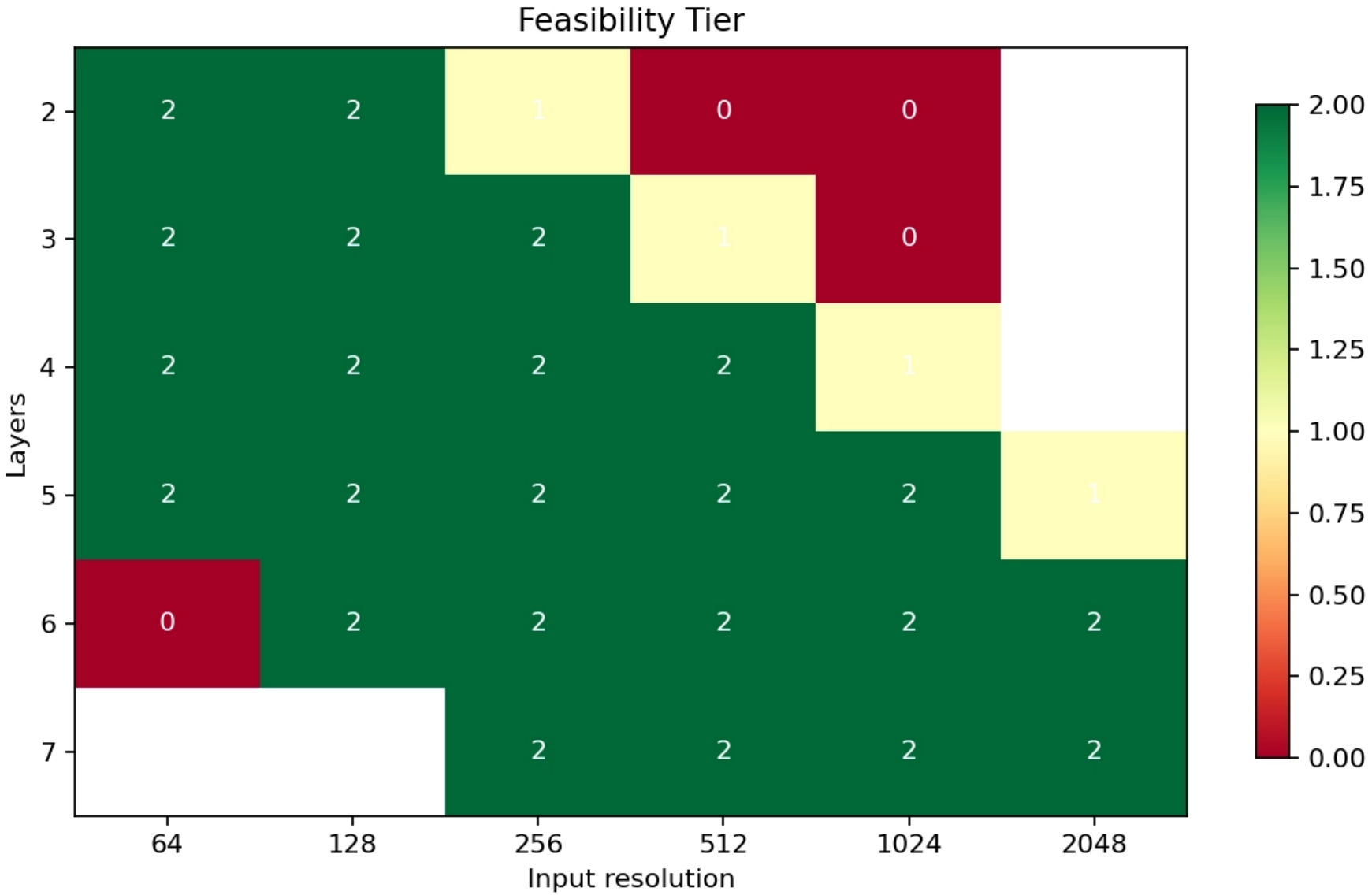
END-TO-END PATH

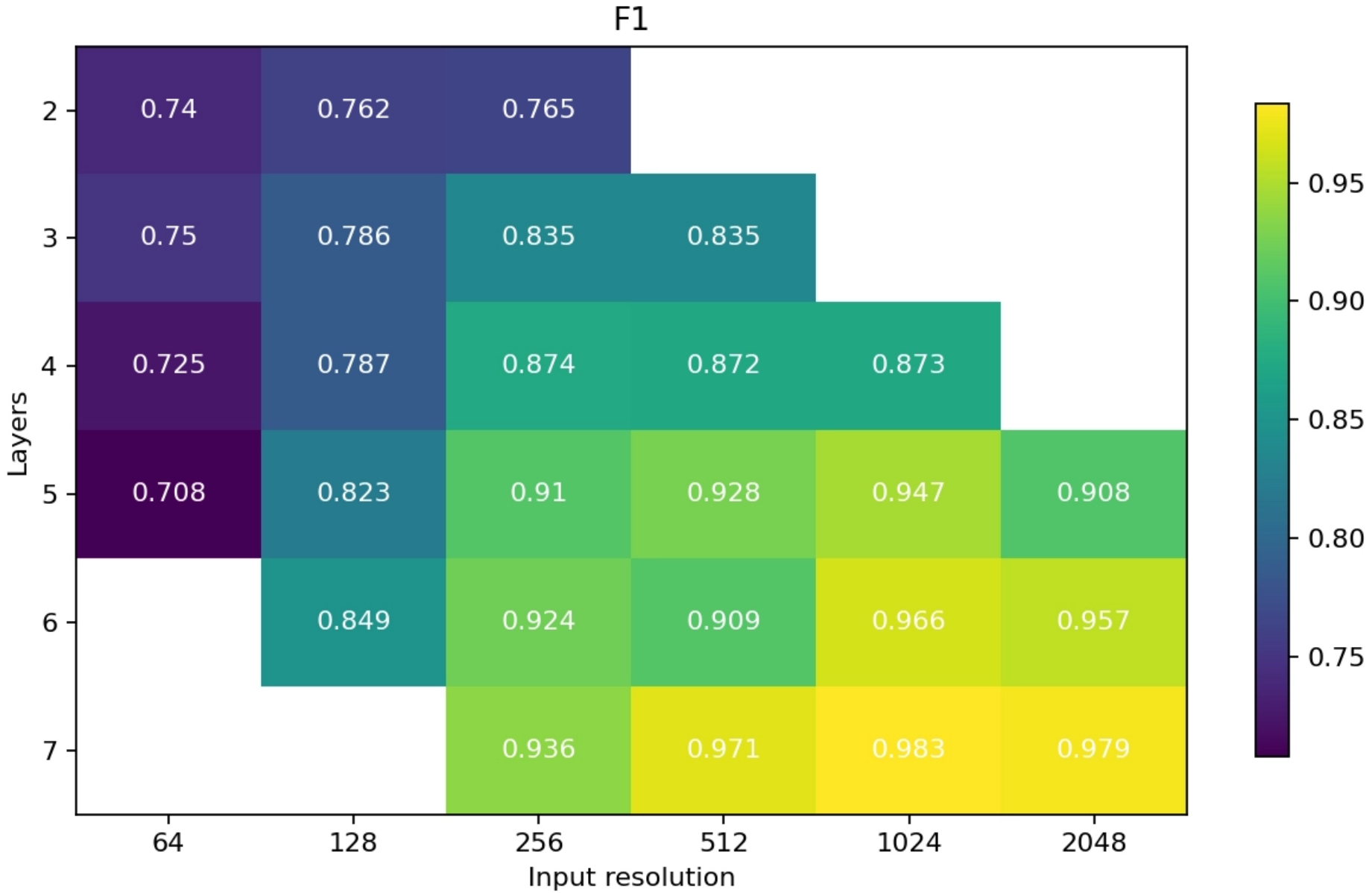


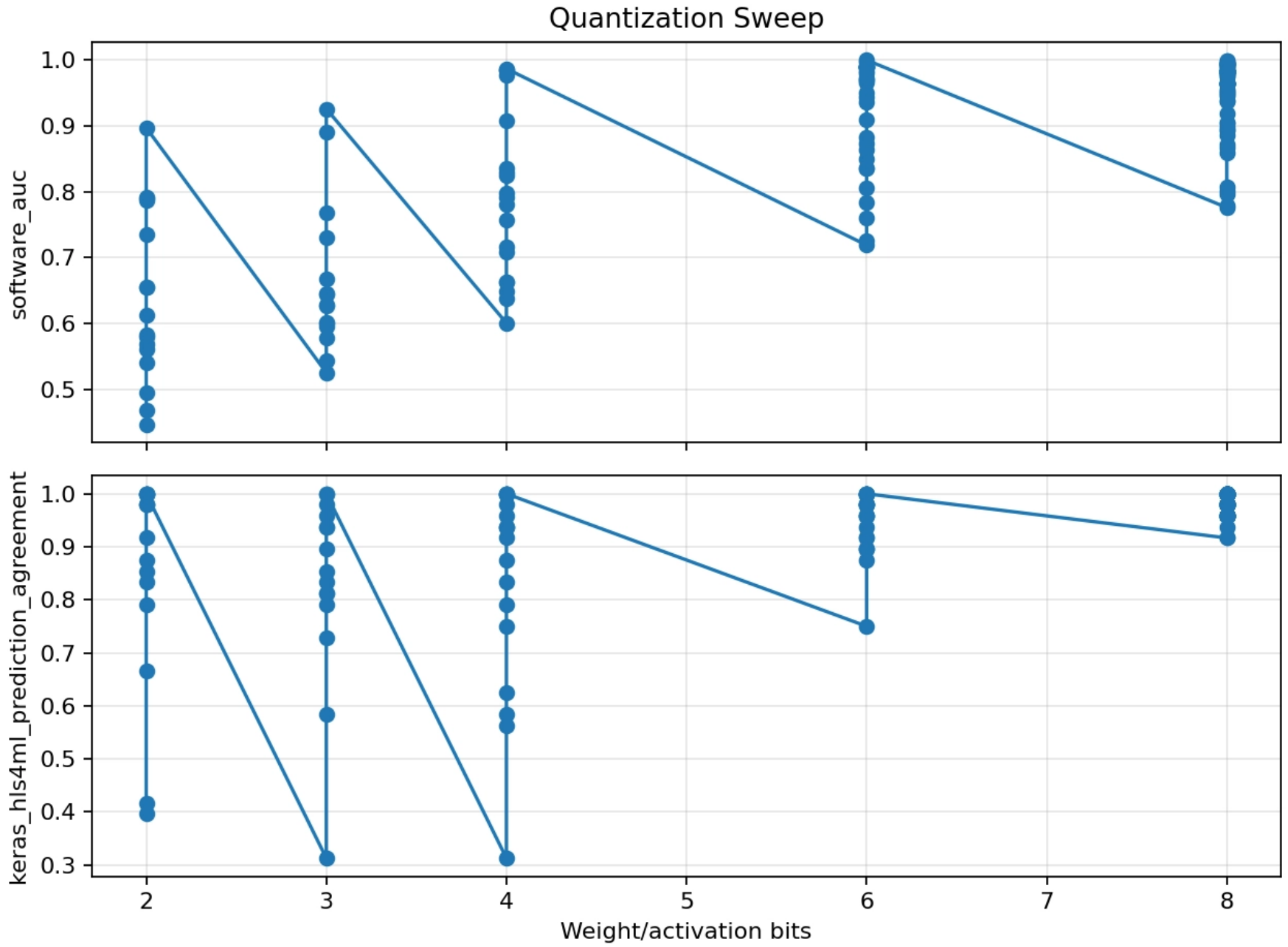
Main takeaway: the prototype shows a practical path from partial bitstreams to an FPGA-only checker before deployment.

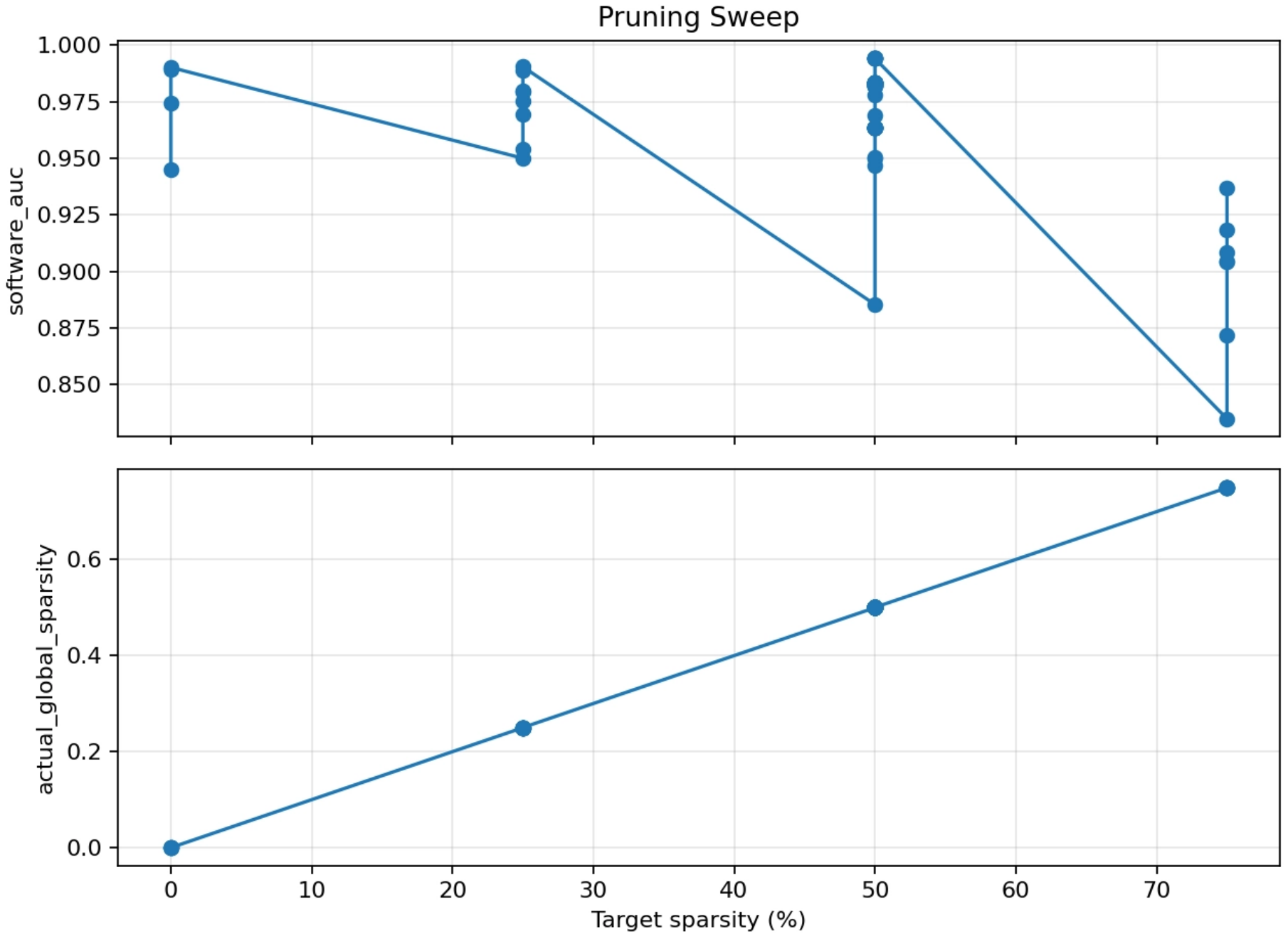

Latency + LUT Combined View
Cycles vs LUT Count
1e6
1.2
1.0
0.8
0.6
0.4
0.2
0.0
LUT count
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
1e7
Latency cycles
Resolution
64px
128px
256px
512px
1024px
2048px
Milliseconds vs LUT %
LUT utilization (%)
80
60
40
20
0
0
20
40
60
80
Latency (ms @ 5.0 ns)


Best Achievable F1 as LUT Budget Increases
Maximum F1
LUT utilization (% of device)
res512_layers7_W8A8_P50_RFResource32
res256_layers7_W8A8_P50_RFResource32
res1024_layers7_W6A6_P0
res1024_layers6_W8A8_P0
res1024_layers7_W8A8_P0
res1024_layers7_WfloatAfloat_P0
64px
128px
256px
512px
1024px
2048px
Best F1 at or below LUT %

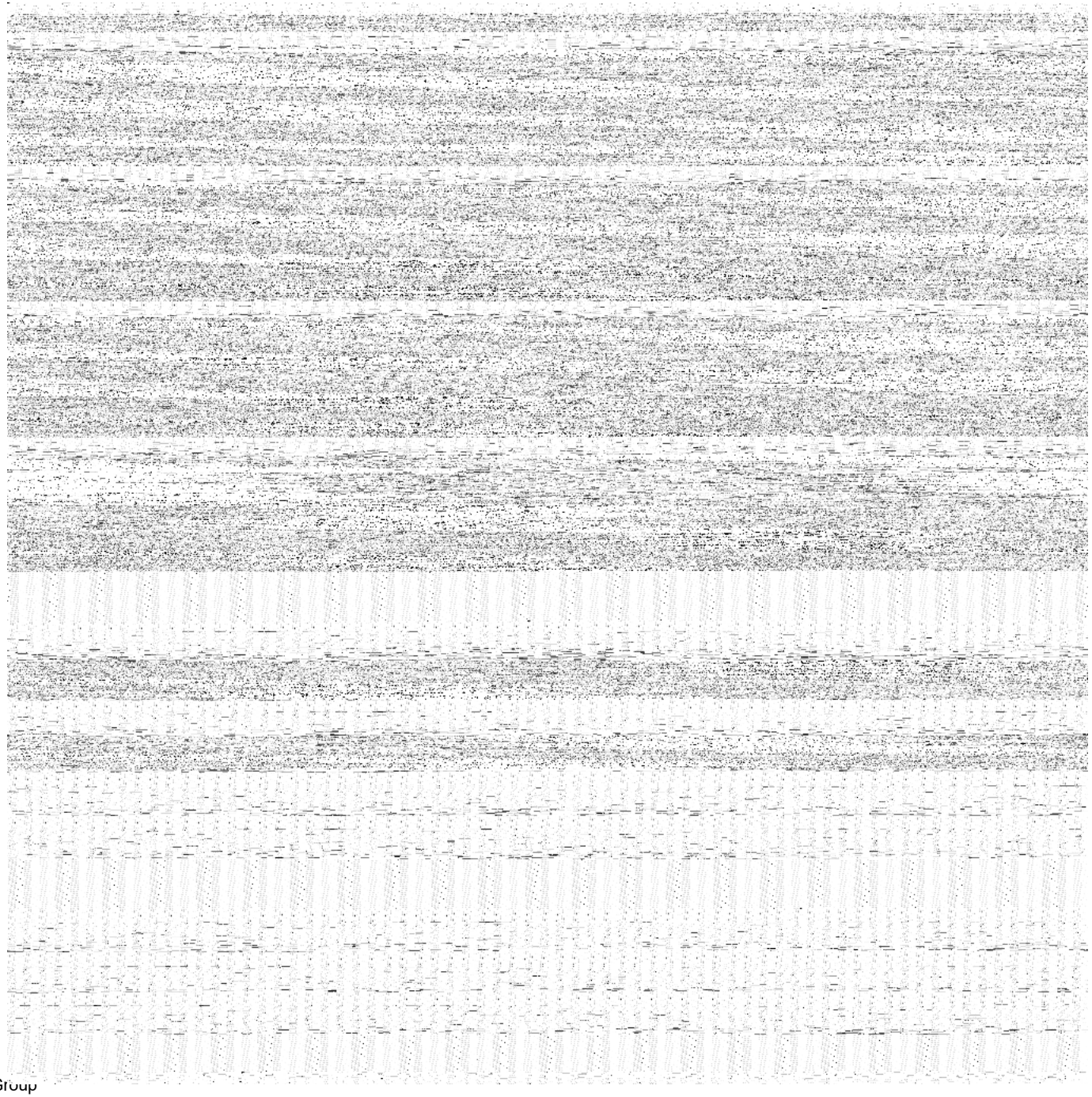

Feasibility Tier
Layers
Input resolution
2
3
4
5
6
7
64
128
256
512
1024
2048
2 2 1 0 0
2 2 2 1 0
2 2 2 2 1
2 2 2 2 2 1
0 2 2 2 2 2
2 2 2 2
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00


F1
Layers
Input resolution
2
3
4
5
6
7
64
128
256
512
1024
2048
0.74
0.762
0.765
0.75
0.786
0.835
0.835
0.725
0.787
0.874
0.872
0.873
0.708
0.823
0.91
0.928
0.947
0.908
0.849
0.924
0.909
0.966
0.957
0.936
0.971
0.983
0.979
0.95
0.90
0.85
0.80
0.75

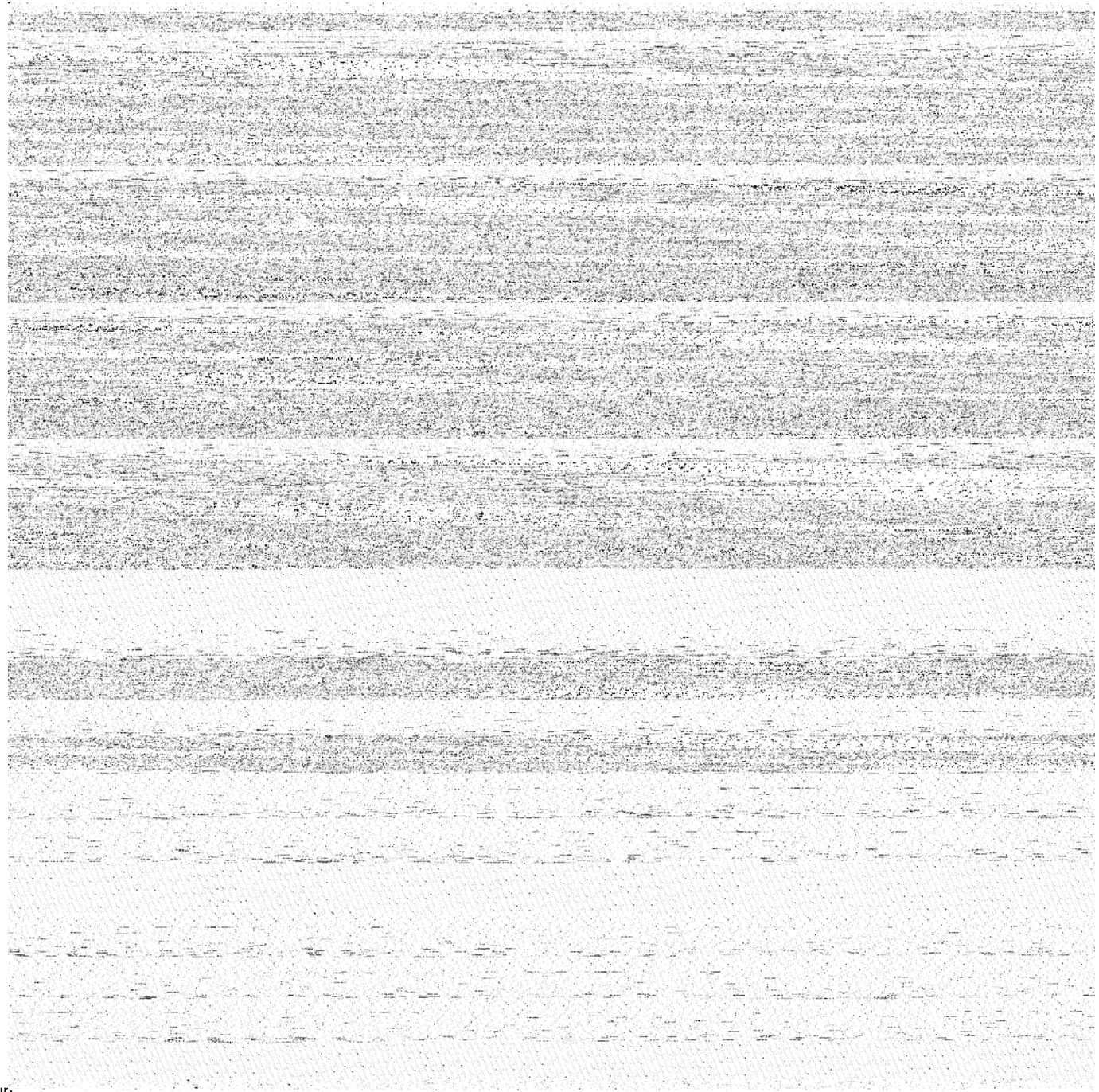

Quantization Sweep
software_auc
keras_hls4ml_prediction_agreement
Weight/activation bits
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
2
3
4
5
6
7
8


Pruning Sweep
software_auc
1.000
0.975
0.950
0.925
0.900
0.875
0.850
actual_global_sparsity
0.6
0.4
0.2
0.0
0
10
20
30
40
50
60
70
Target sparsity (%)

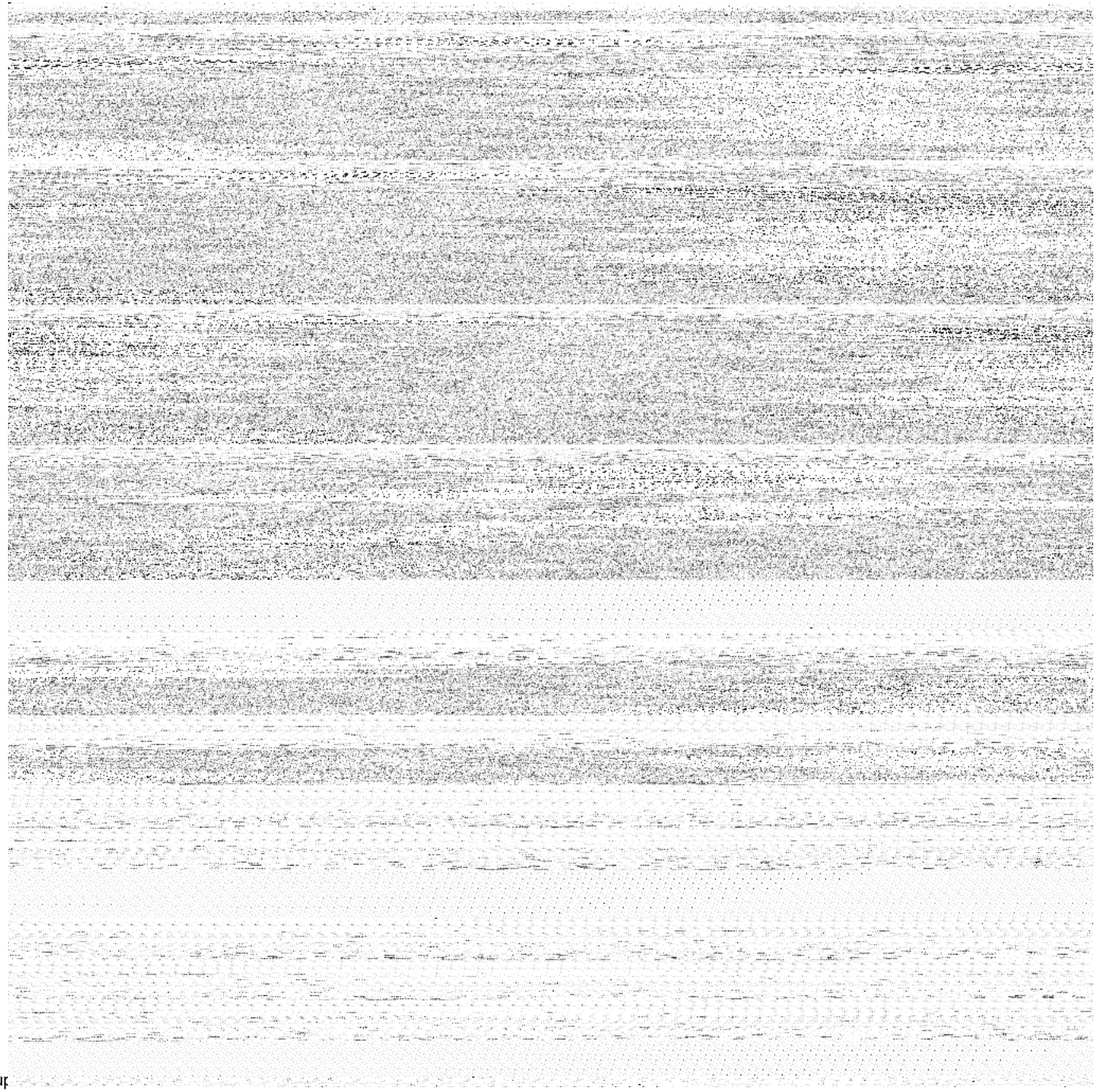
Multithreading AES (benign)



RO=~8000

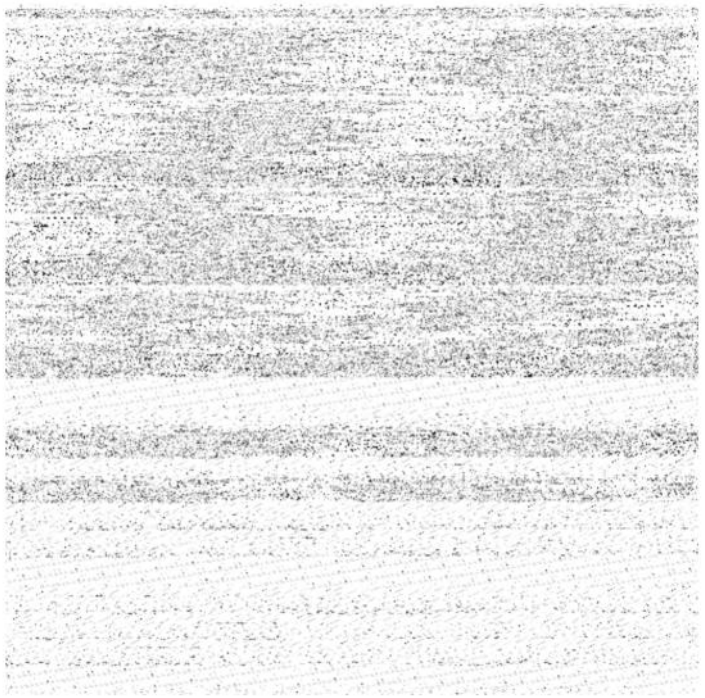


RO=22,000

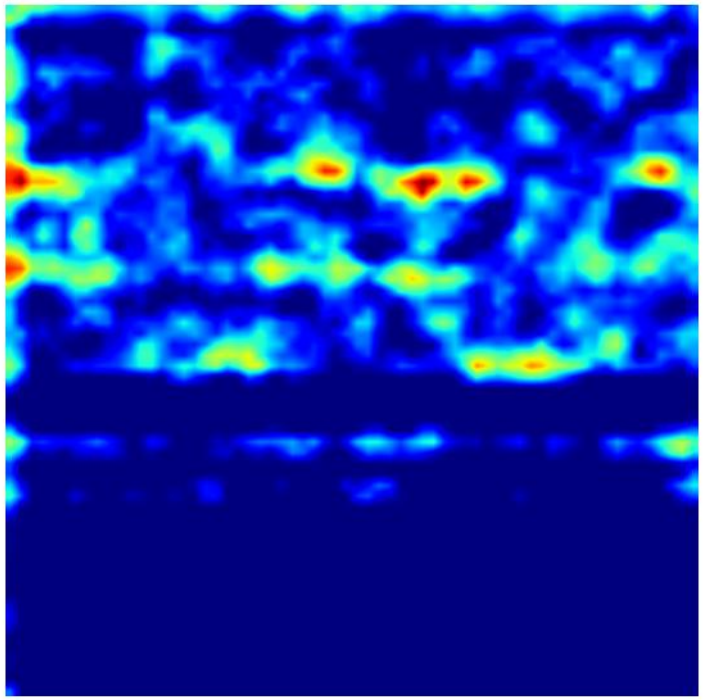


it2_B051 app=euclidean true=benign p=0.4202 target=standalone layer=conv2 64x64x24 fold_0

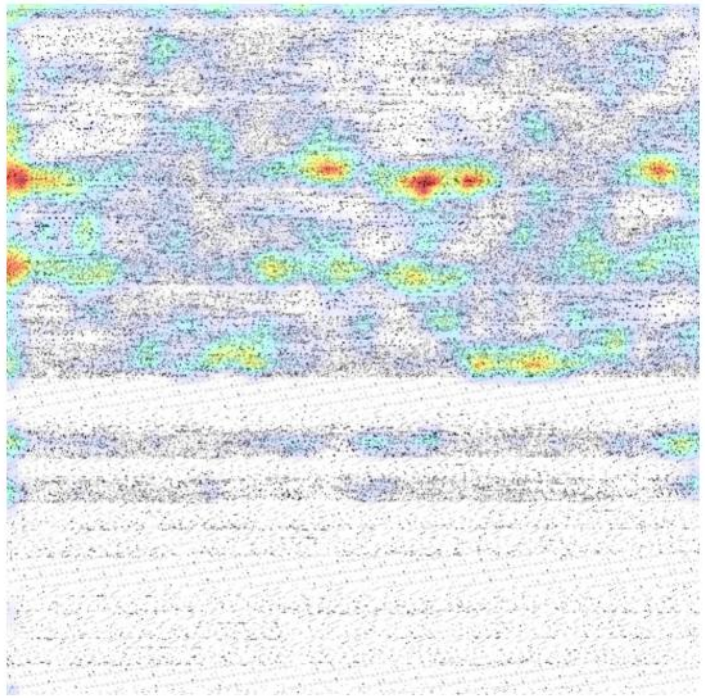
Input



CAM: standalone

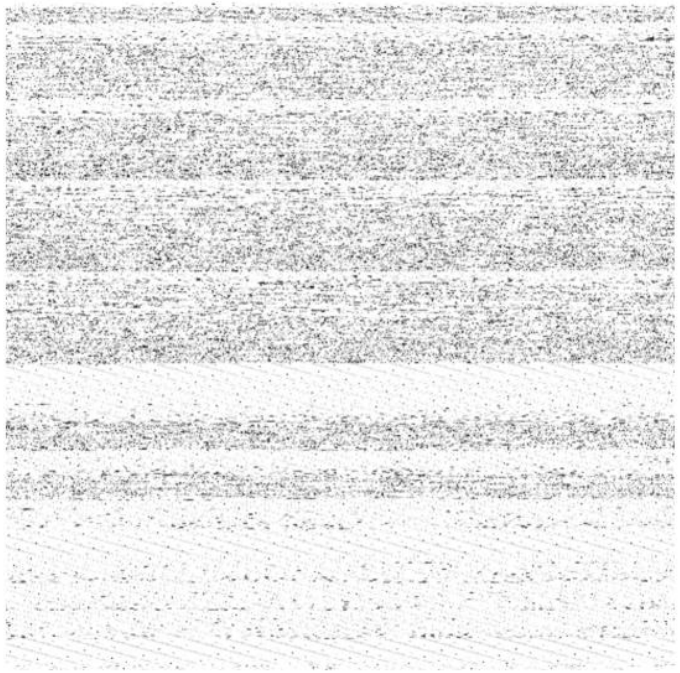


Overlay

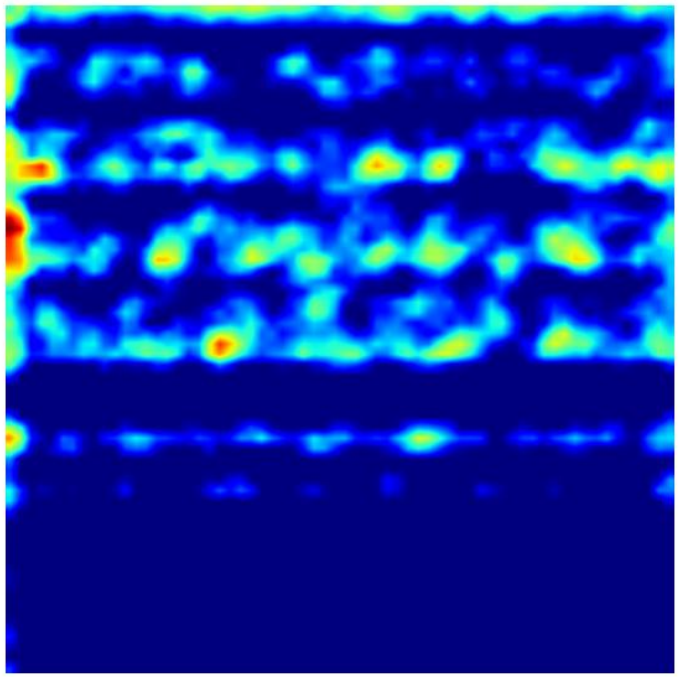


it1_B030 app=hello_world true=benign p=0.0049 target=standalone layer=conv2 64x64x24 fold_0

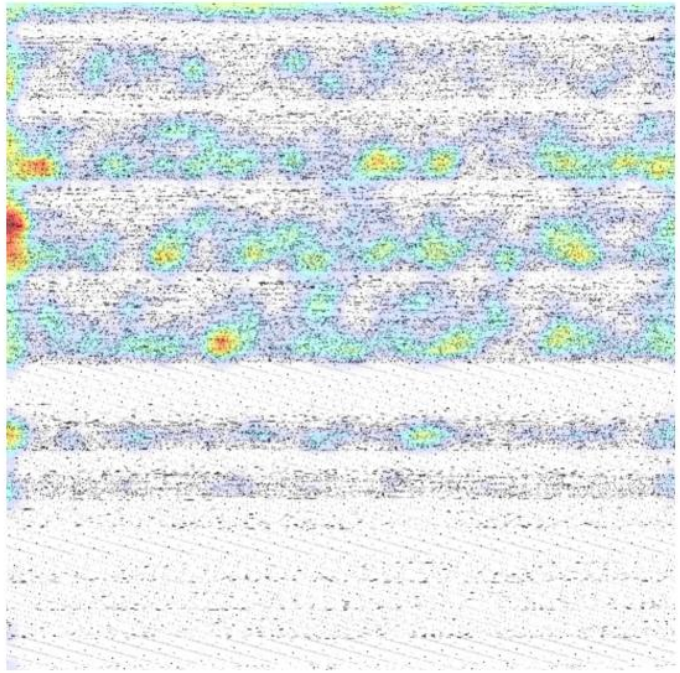
Input



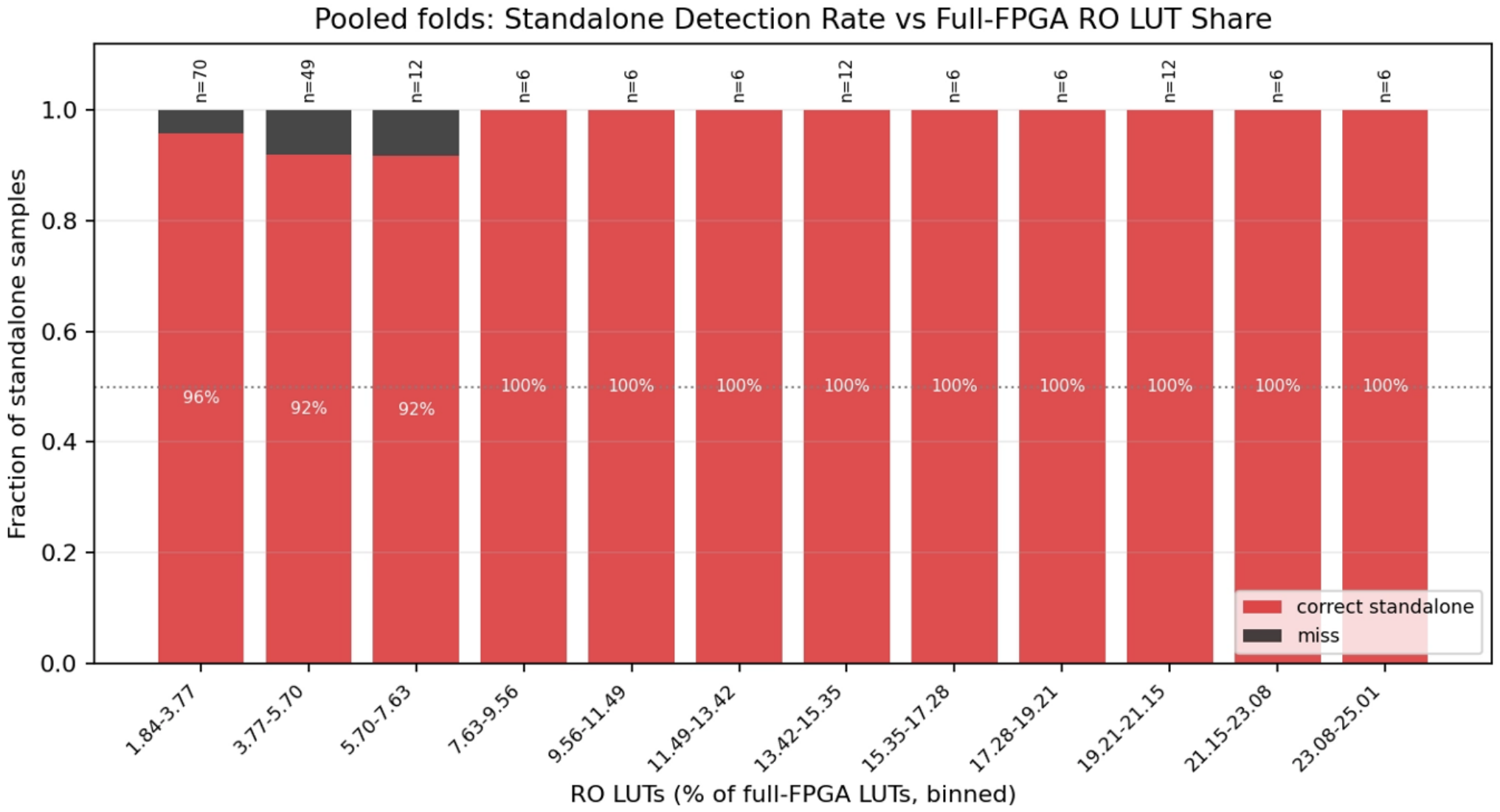
CAM: standalone



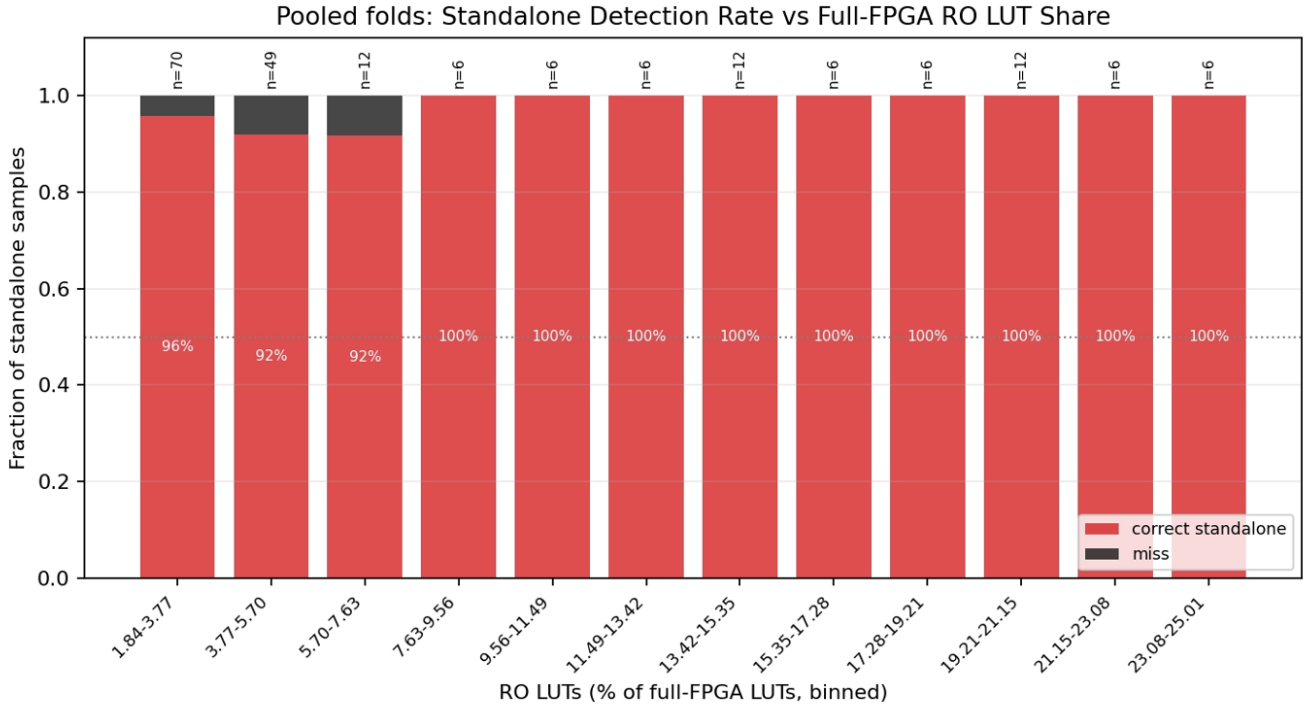
Overlay

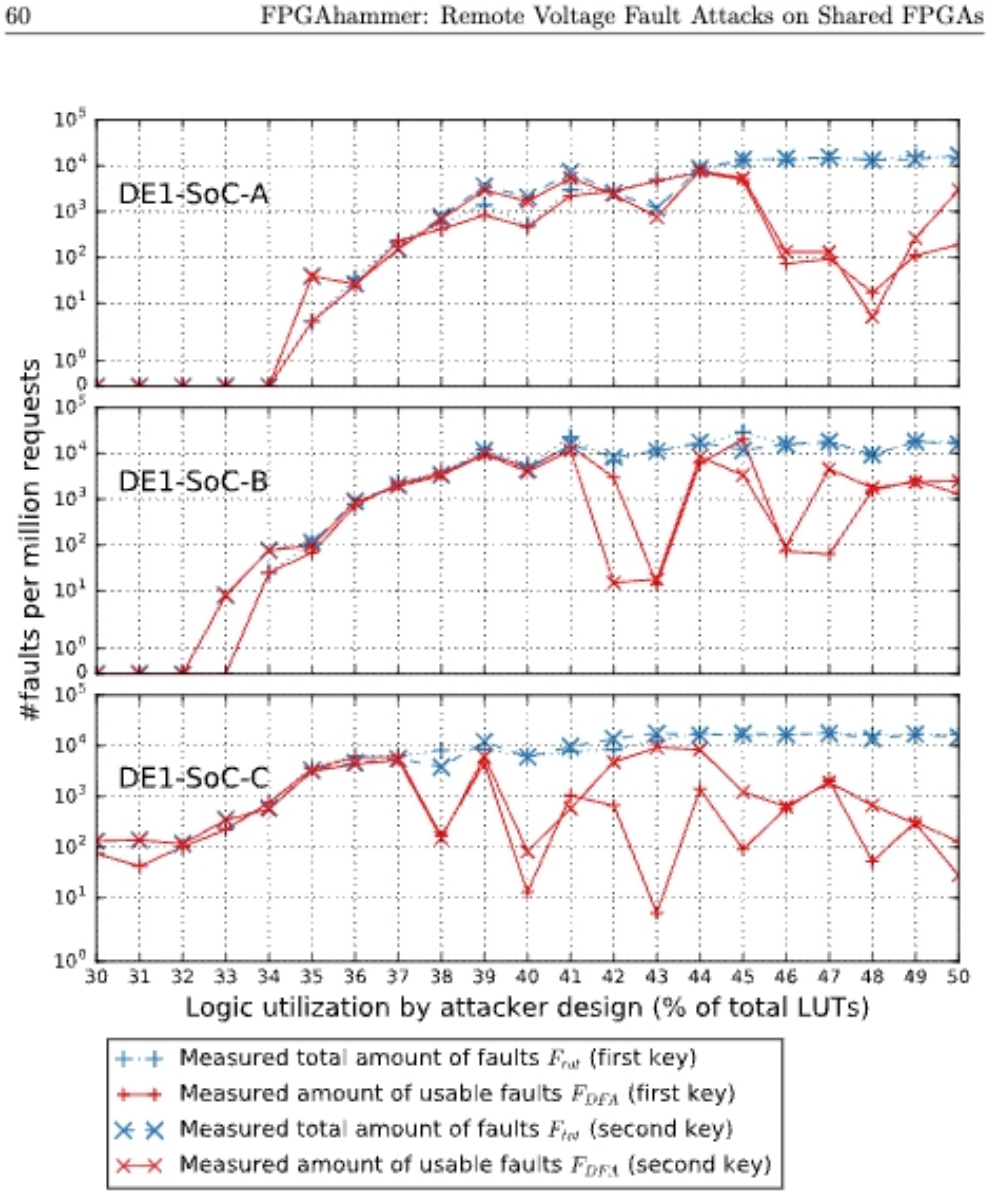


Effect of RO Footprint on Detection: 256-resolution

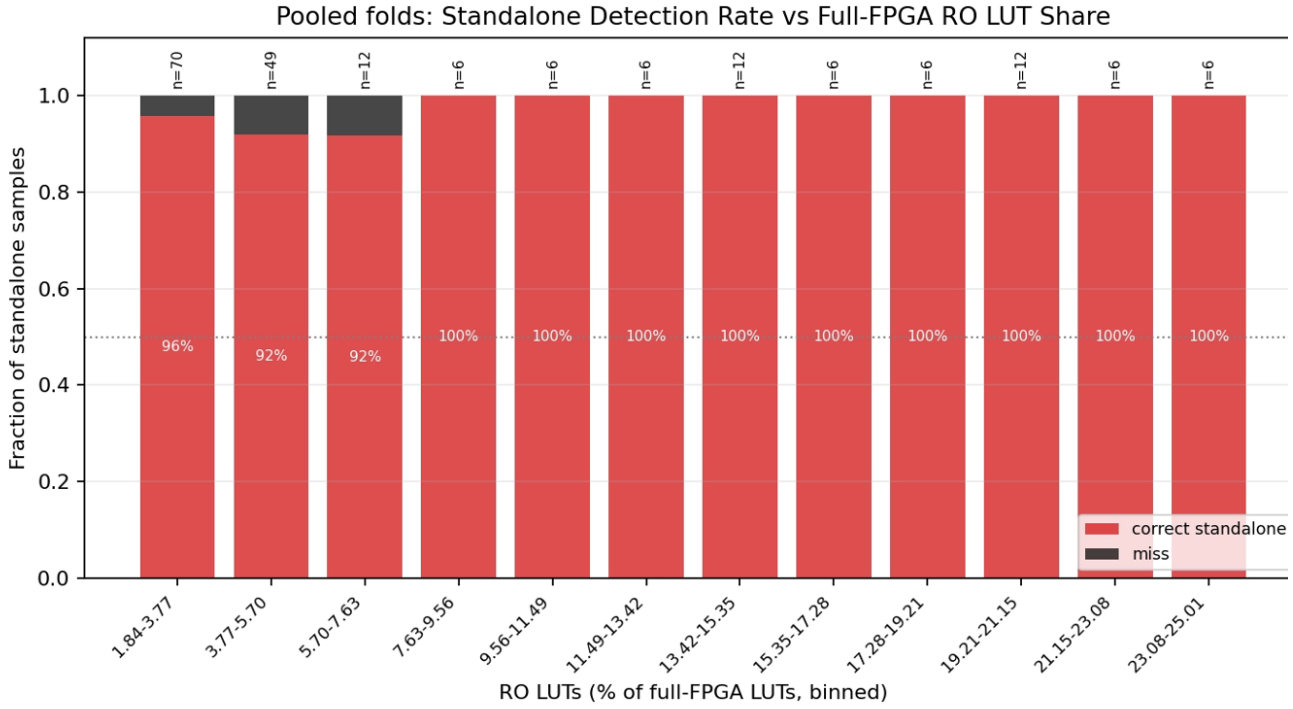


Effect of RO Footprint on Detection: 256-resolution





Effect of RO Footprint on Detection: 256-resolution



T. M. La et al.

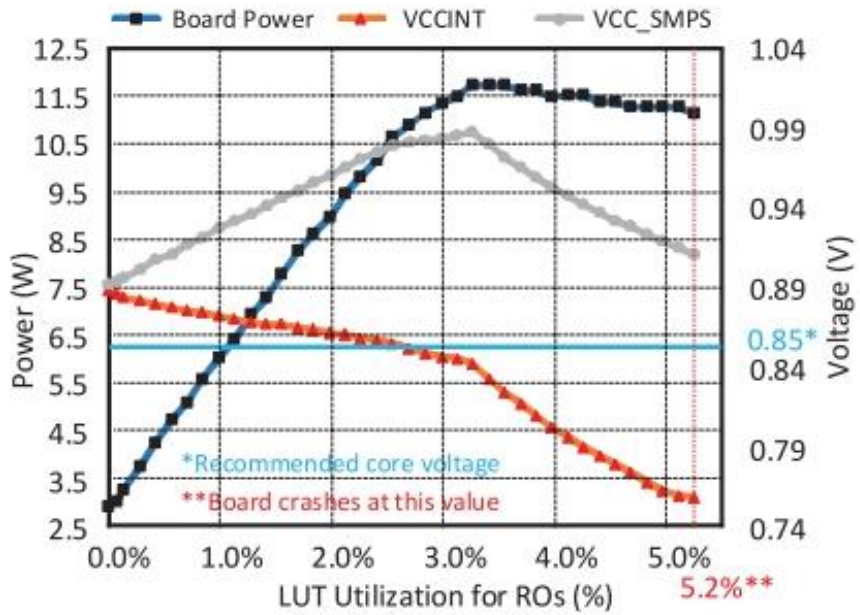
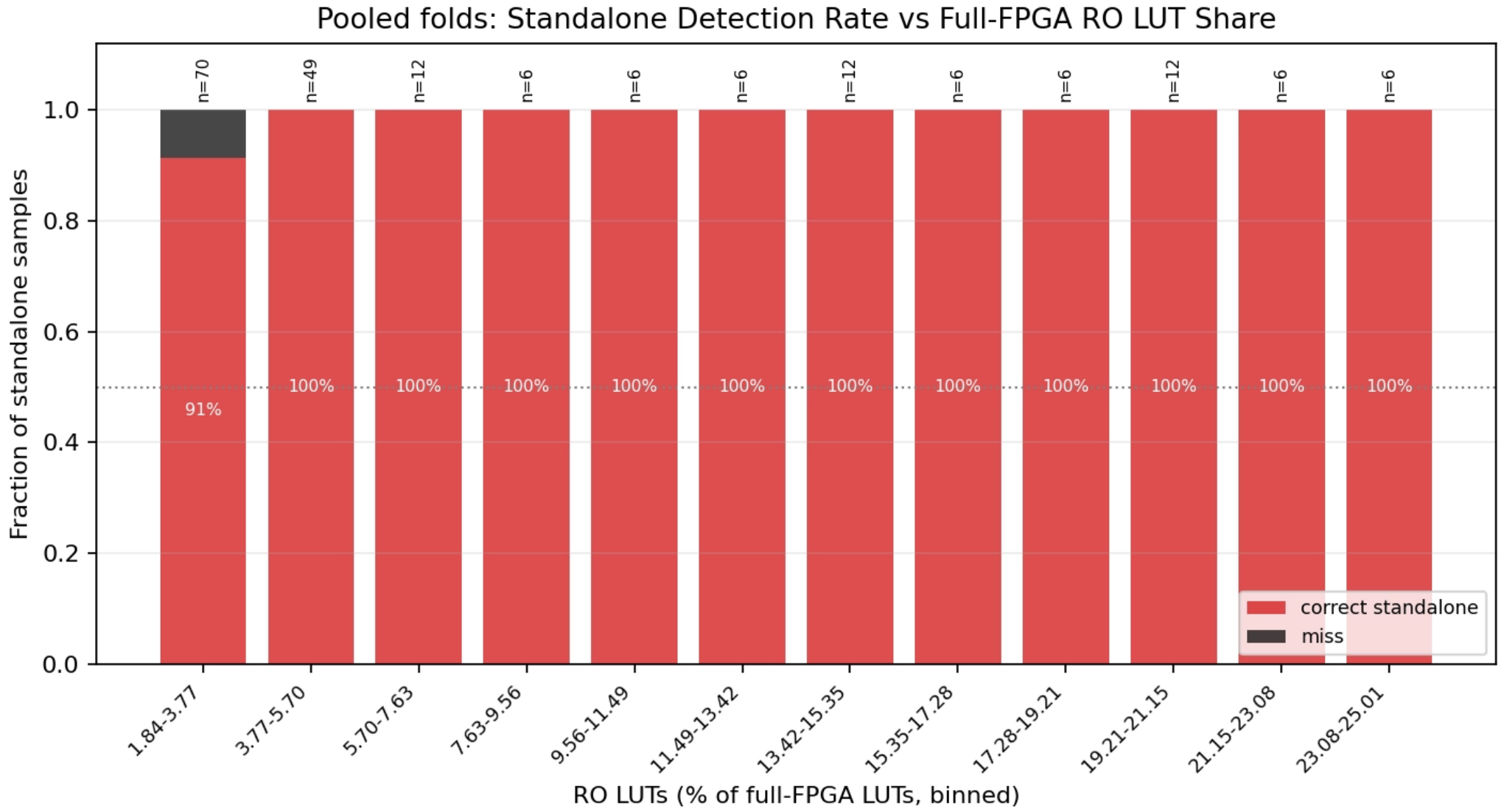
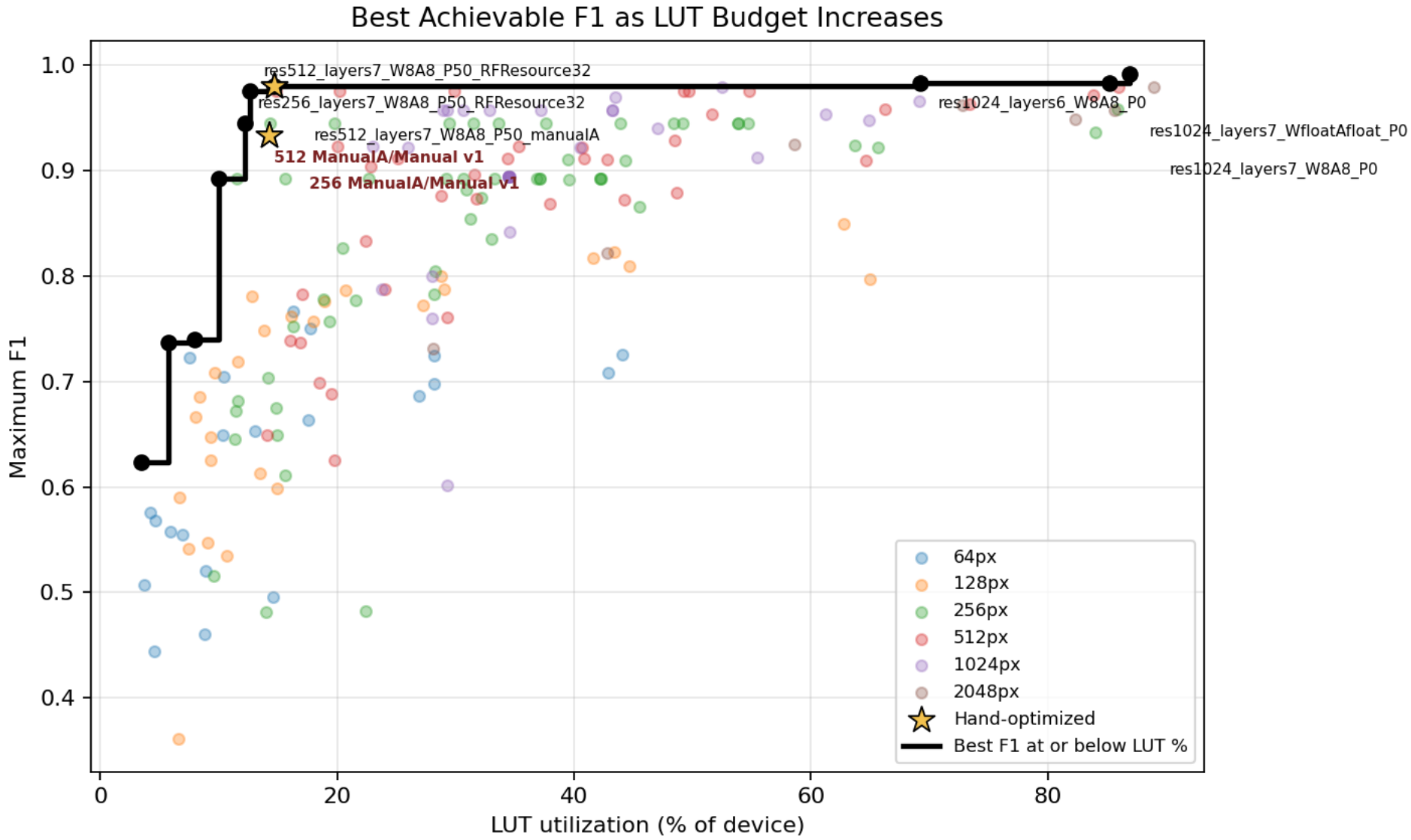


Fig. 13. Power-hammering Evaluation for Power over Core Voltage on Ultra96.

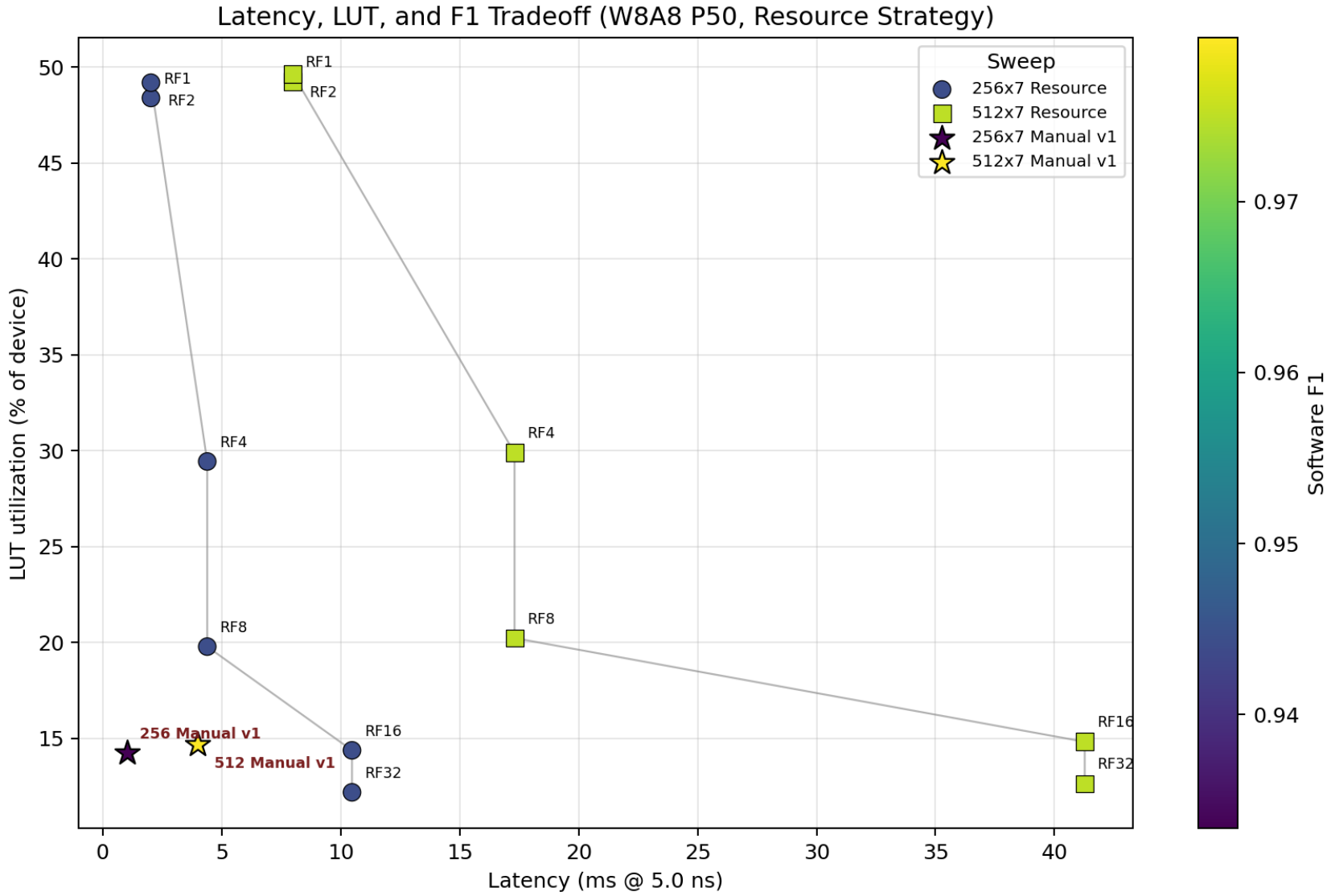
Effect of RO Footprint on Detection: 512-resolution



HLS Exploration: Latency vs. Resource vs. Accuracy [F1] Trade-off (2)



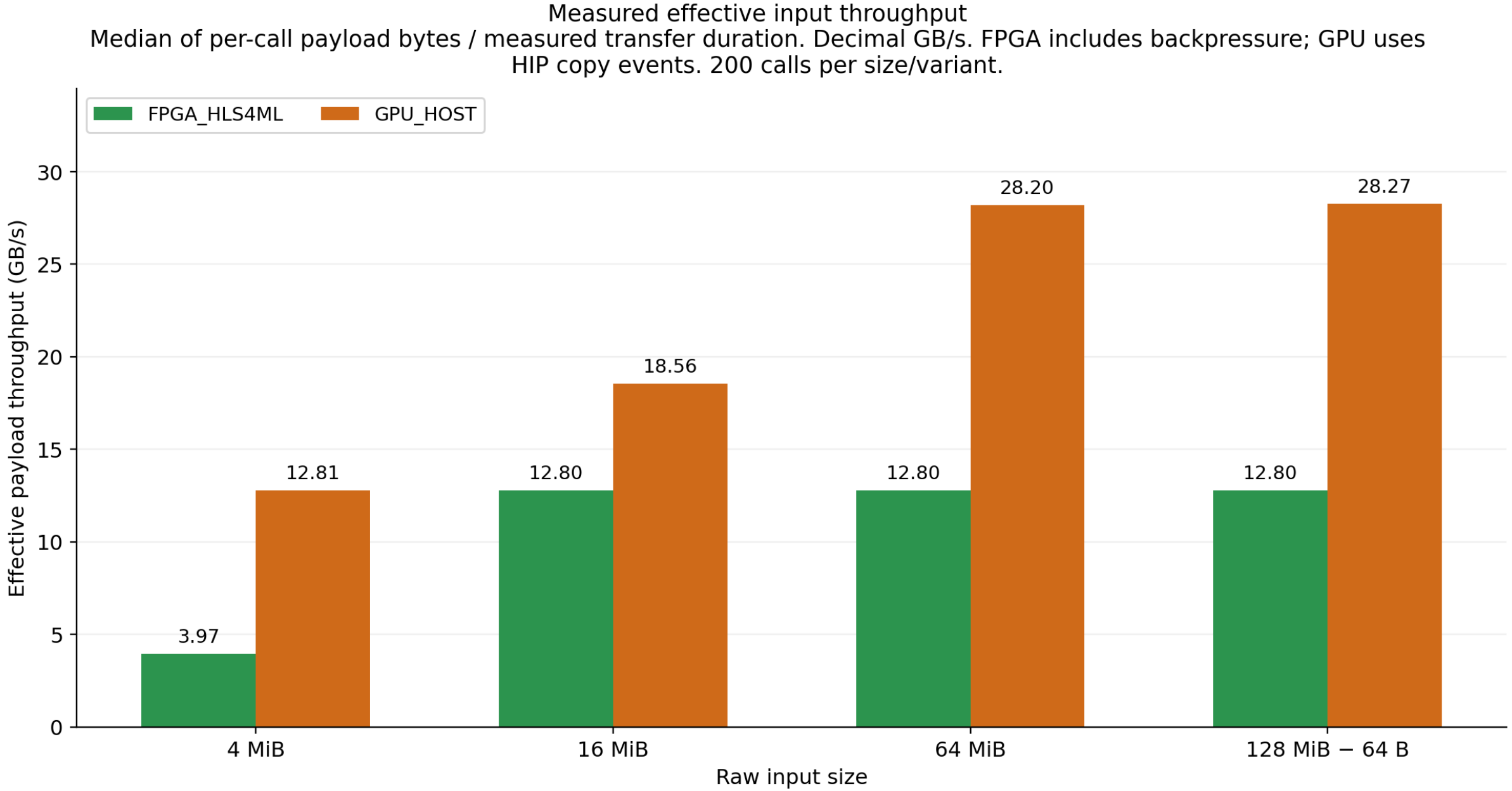
HLS Exploration: Latency vs. Resource vs. Accuracy [F1] Trade-off



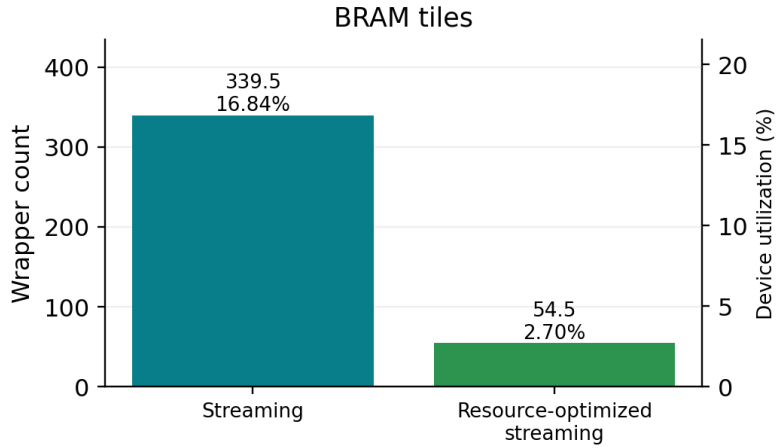
Baseline Coyote Resources

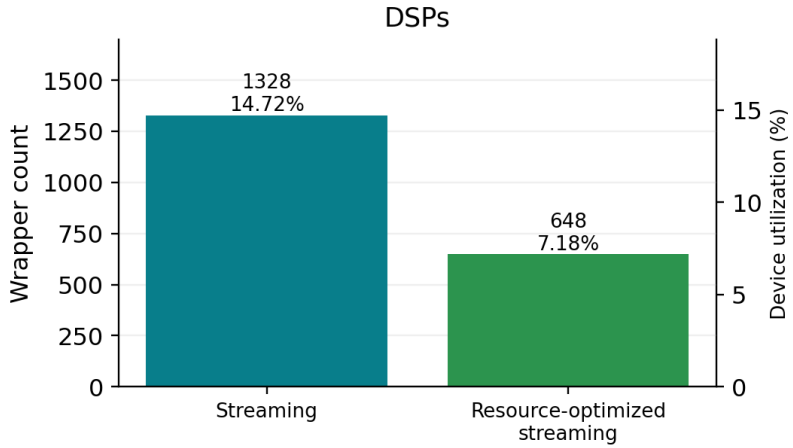
Model	Added latency*	Added LUT†	Added BRAM†	Added DSP†
256×7	**3.35 ms**	**+4.47%**	**+17.86%**	+14.72%
512×7	6.53 ms	+4.90%	+61.78%	**+14.45%**

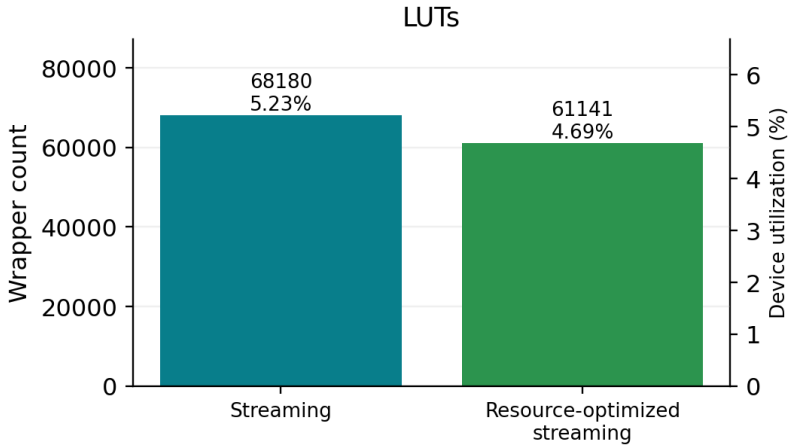
Model	LUT	BRAM†	URAM	DSP
Baseline Coyote	10.26%	9.77%	0%	0%

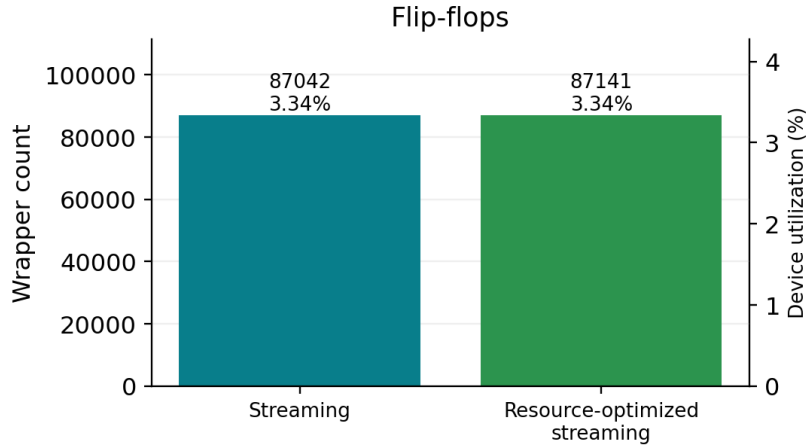

Measured effective input throughput
Median of per-call payload bytes / measured transfer duration. Decimal GB/s. FPGA includes backpressure; GPU uses HIP copy events. 200 calls per size/variant.
FPGA_HLS4ML
GPU_HOST
Effective payload throughput (GB/s)
0
5
10
15
20
25
30
3.97
12.81
12.80
18.56
12.80
28.20
12.80
28.27
4 MiB
16 MiB
64 MiB
128 MiB − 64 B
Raw input size

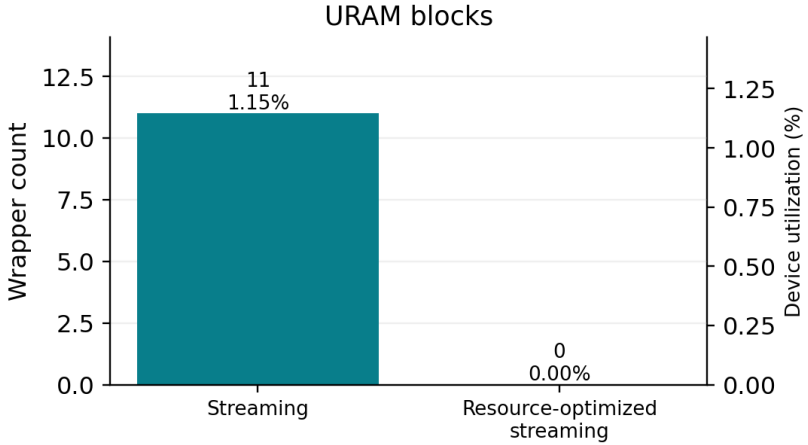
Checker optimization: FPGA wrapper resource utilization
Routed user wrapper, including interfaces and ILA; percentages use total U55C capacity. Shell resources excluded.







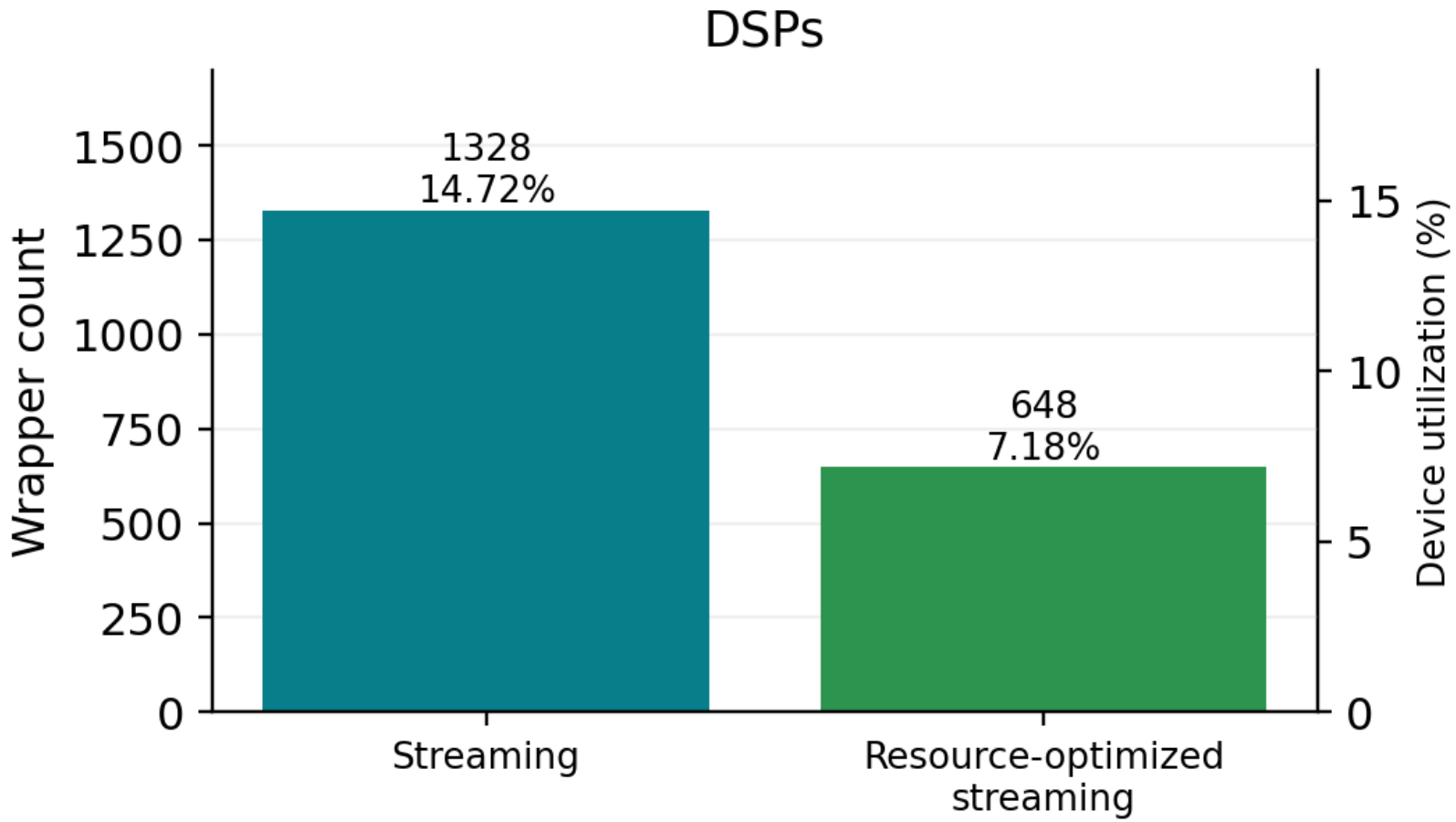


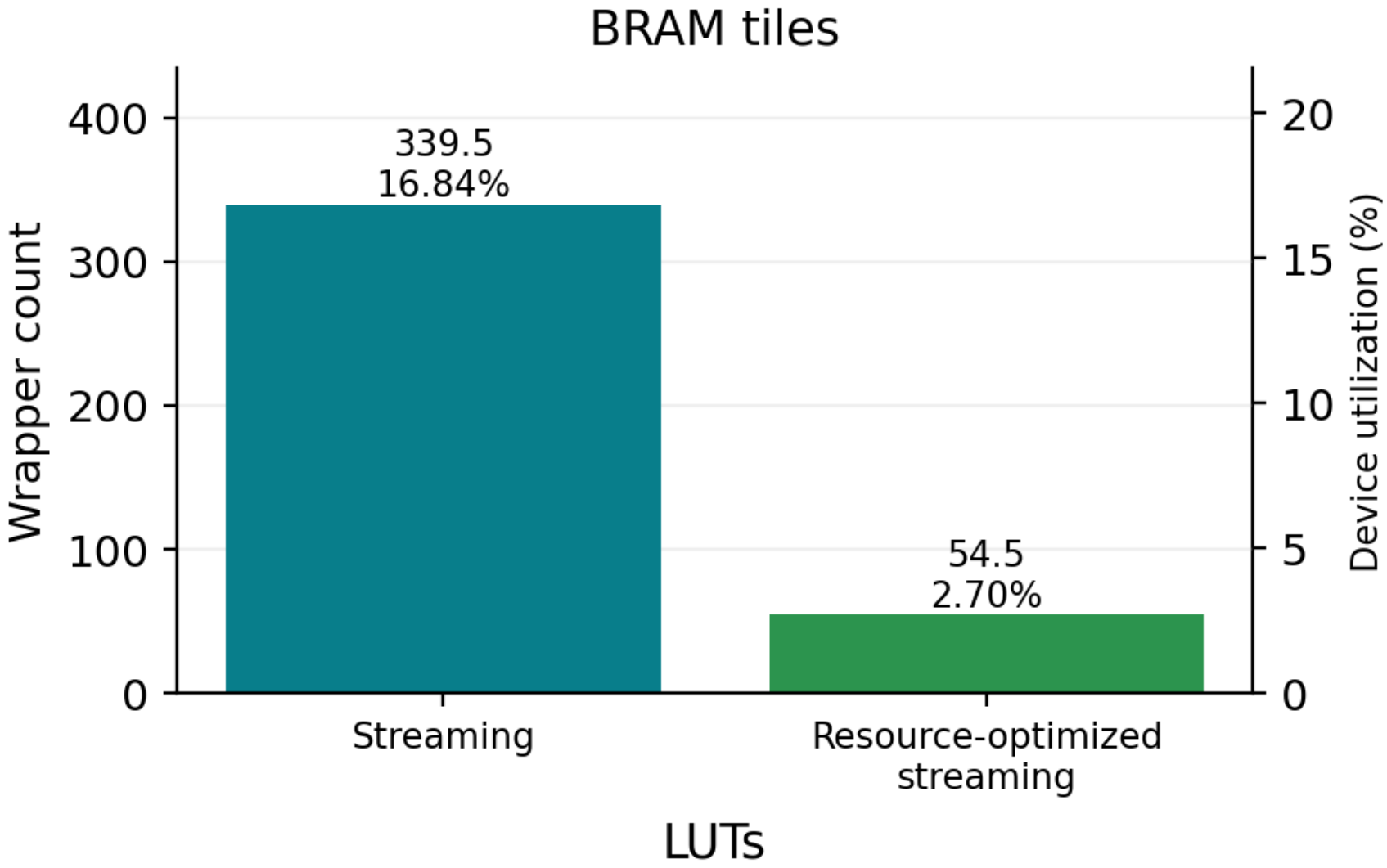


Resource-optimized streaming versus Streaming

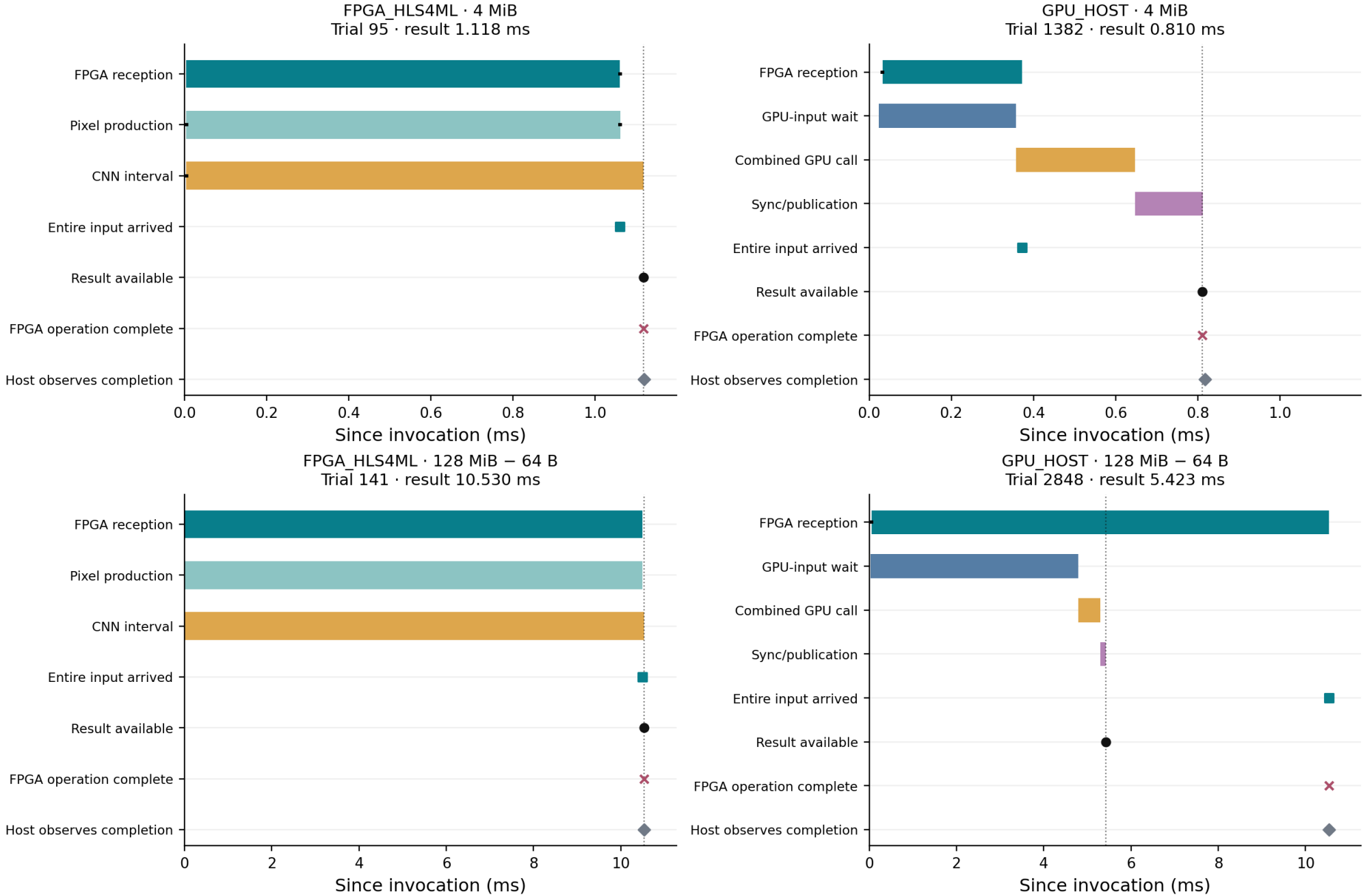
BRAM: −83.95%
DSP: −51.20%
URAM: 11 → 0

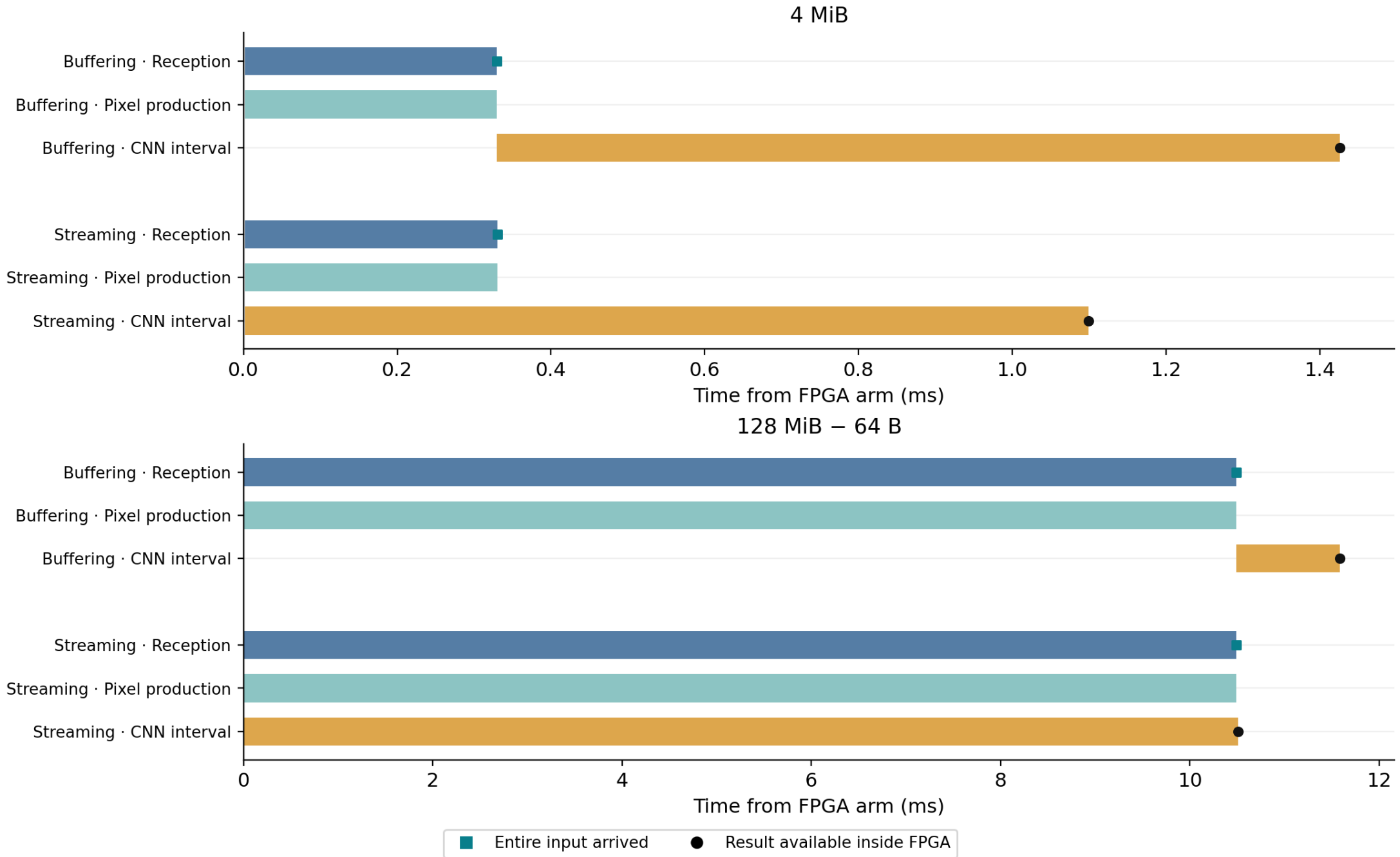
Routed wrapper includes interfaces and ILA.

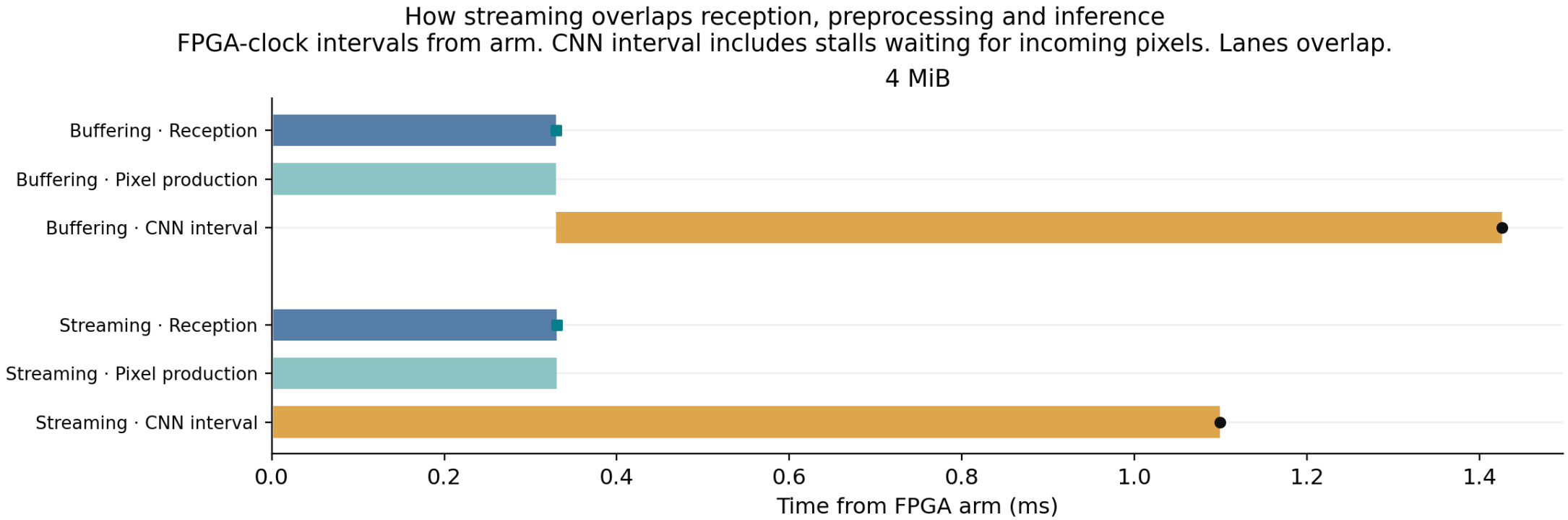

DSPs
Wrapper count
1500
1250
1000
750
500
250
0
1328
14.72%
648
7.18%
15
10
5
0
Device utilization (%)
Streaming
Resource-optimized
streaming


BRAM tiles
Wrapper count
Device utilization (%)
400
300
200
100
0
20
15
10
5
0
339.5
16.84%
54.5
2.70%
Streaming
Resource-optimized streaming
LUTs

Measured execution overlap and FPGA result availability
Actual trials nearest median result availability. Parallel lanes overlap; do not add their durations. GPU: optimized with XLA compilation + ConcreteFunction FP(32); FPGA: W8A8.




How streaming overlaps reception, preprocessing and inference
FPGA-clock intervals from arm. CNN interval includes stalls waiting for incoming pixels. Lanes overlap.
4 MiB
Buffering · Reception
Buffering · Pixel production
Buffering · CNN interval
Streaming · Reception
Streaming · Pixel production
Streaming · CNN interval
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Time from FPGA arm (ms)
128 MiB − 64 B
Buffering · Reception
Buffering · Pixel production
Buffering · CNN interval
Streaming · Reception
Streaming · Pixel production
Streaming · CNN interval
0
2
4
6
8
10
12
Time from FPGA arm (ms)
Entire input arrived
Result available inside FPGA


How streaming overlaps reception, preprocessing and inference
FPGA-clock intervals from arm. CNN interval includes stalls waiting for incoming pixels. Lanes overlap.
4 MiB
Buffering · Reception
Buffering · Pixel production
Buffering · CNN interval
Streaming · Reception
Streaming · Pixel production
Streaming · CNN interval
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Time from FPGA arm (ms)

Verdict availability and full-input arrival at the FPGA
Means of 200 calls; reception lanes overlap the stacked intervals. 128* = 128 MiB − 64 B. GPU: optimized with XLA compilation + ConcreteFunction FP(32); FPGA: W8A8.

